

# Java Stream API Coding Interview Questions

---

## 1. Basic Stream Operations

---

### Creating & Converting Streams

**Q1. Given a list of integers, convert it to a stream and print all elements.**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
numbers.stream().forEach(n -> System.out.println(n));
// Or using method reference:
numbers.stream().forEach(System.out::println);
```

---

**Q2. Create a stream from an array of strings.**

```
String[] arr = {"apple", "banana", "cherry"};
Stream<String> stream = Arrays.stream(arr);
stream.forEach(System.out::println);
```

---

**Q3. Create an infinite stream of random numbers and limit it to 10 elements.**

```
Stream.generate(Math::random)
    .limit(10)
    .forEach(System.out::println);
```

---

**Q4. Create a stream of integers from 1 to 100.**

```
IntStream.rangeClosed(1, 100)
    .forEach(System.out::println);
```

---

**Q5. Convert an array to a list using streams.**

```
String[] arr = {"a", "b", "c"};
List<String> list = Arrays.stream(arr).collect(Collectors.toList());
```

---

**Q6. Create a stream from a Map's keys.**

```
Map<String, Integer> map = Map.of("a", 1, "b", 2);
map.keySet().stream().forEach(System.out::println);
```

---

**Q7. Create a stream from a Map's values.**

```
Map<String, Integer> map = Map.of("a", 1, "b", 2);
map.values().stream().forEach(System.out::println);
```

---

**Q8. Create a stream from a Map's entries.**

```
Map<String, Integer> map = Map.of("a", 1, "b", 2);
map.entrySet().stream().forEach(e -> System.out.println(e.getKey() + "=" + e.getValue()));
```

---

**Q9. Create a stream of custom objects (e.g., Employee, Student).**

```
List<Employee> employees = Arrays.asList(
    new Employee("Alice", 50000),
    new Employee("Bob", 60000)
);
employees.stream().forEach(System.out::println);
```

---

**Q10. Convert a Set to a List using streams.**

```
Set<String> set = Set.of("x", "y", "z");
List<String> list = set.stream().collect(Collectors.toList());
```

---

## Counting & Iteration

**Q11. Count the total number of elements in a list using streams.**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
long count = numbers.stream().count();
System.out.println(count); // 5
```

---

**Q12. Print all elements in a list using forEach.**

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
names.stream().forEach(System.out::println);
```

---

**Q13. Count the number of elements greater than a certain value.**

```
List<Integer> numbers = Arrays.asList(5, 10, 15, 20, 25);
long count = numbers.stream().filter(n -> n > 10).count();
System.out.println(count); // 3
```

---

**Q14. Count how many strings start with a specific character.**

```
List<String> names = Arrays.asList("Alice", "Bob", "Anna", "Charlie");
long count = names.stream().filter(s -> s.startsWith("A")).count();
System.out.println(count); // 2
```

---

**Q15. Iterate over a stream and perform operations on each element.**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
numbers.stream()
    .map(n -> n * 2)
    .forEach(System.out::println); // 2, 4, 6, 8, 10
```

---

## 2. Filtering & Mapping

### Filtering Operations

**Q16. Filter all even numbers from a list of integers.**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);
List<Integer> evens = numbers.stream()
    .filter(n -> n % 2 == 0)
    .collect(Collectors.toList());

// [2, 4, 6]
```

---

**Q17. Filter all odd numbers from a list of integers.**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);
List<Integer> odds = numbers.stream()
    .filter(n -> n % 2 != 0)
    .collect(Collectors.toList());

// [1, 3, 5]
```

---

**Q18. Filter strings that start with a specific letter.**

```
List<String> names = Arrays.asList("Alice", "Bob", "Anna", "Charlie");
List<String> result = names.stream()
    .filter(s -> s.startsWith("A"))
    .collect(Collectors.toList());

// [Alice, Anna]
```

---

**Q19. Filter strings that end with a specific letter.**

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie", "Mike");
List<String> result = names.stream()
    .filter(s -> s.endsWith("e"))
    .collect(Collectors.toList());

// [Alice, Charlie, Mike]
```

---

**Q20. Filter strings that contain a specific substring.**

```
List<String> words = Arrays.asList("apple", "application", "banana", "appetite");
List<String> result = words.stream()
    .filter(s -> s.contains("app"))
    .collect(Collectors.toList());

// [apple, application, appetite]
```

---

**Q21. Filter numbers greater than a specific value.**

```
List<Integer> numbers = Arrays.asList(3, 7, 12, 18, 25);
List<Integer> result = numbers.stream()
    .filter(n -> n > 10)
    .collect(Collectors.toList());

// [12, 18, 25]
```

---

**Q22. Filter numbers less than a specific value.**

```
List<Integer> numbers = Arrays.asList(3, 7, 12, 18, 25);
List<Integer> result = numbers.stream()
    .filter(n -> n < 10)
    .collect(Collectors.toList());

// [3, 7]
```

---

**Q23. Filter numbers within a specific range (e.g., between 10 and 50).**

```
List<Integer> numbers = Arrays.asList(5, 15, 30, 55, 70);
List<Integer> result = numbers.stream()
    .filter(n -> n >= 10 && n <= 50)
    .collect(Collectors.toList());

// [15, 30]
```

---

**Q24. Filter null values from a list.**

```
List<String> names = Arrays.asList("Alice", null, "Bob", null, "Charlie");
List<String> result = names.stream()
    .filter(s -> s != null)
    .collect(Collectors.toList());

// [Alice, Bob, Charlie]
```

---

**Q25. Filter empty strings from a list.**

```
List<String> words = Arrays.asList("hello", "", "world", "", "java");
List<String> result = words.stream()
    .filter(s -> !s.isEmpty())
    .collect(Collectors.toList());

// [hello, world, java]
```

---

**Q26. Filter blank/whitespace strings from a list.**

```
List<String> words = Arrays.asList("hello", " ", "world", "\t", "java");
List<String> result = words.stream()
    .filter(s -> !s.isBlank())
    .collect(Collectors.toList());

// [hello, world, java]
```

---

**Q27. Filter elements based on multiple conditions.**

```
List<Integer> numbers = Arrays.asList(1, 4, 9, 12, 15, 20);
List<Integer> result = numbers.stream()
    .filter(n -> n > 5 && n % 2 == 0)
    .collect(Collectors.toList());

// [12, 20]
```

---

**Q28. Filter numbers divisible by both 3 and 5.**

```
List<Integer> numbers = Arrays.asList(5, 10, 15, 20, 30, 45);
List<Integer> result = numbers.stream()
    .filter(n -> n % 3 == 0 && n % 5 == 0)
    .collect(Collectors.toList());

// [15, 30, 45]
```

---

**Q29. Filter strings with length greater than 5.**

```
List<String> words = Arrays.asList("hi", "hello", "stream", "java", "programming");
List<String> result = words.stream()
    .filter(s -> s.length() > 5)
    .collect(Collectors.toList());

// [stream, programming]
```

---

**Q30. Filter strings with length equal to a specific value.**

```
List<String> words = Arrays.asList("hi", "hey", "hello", "java", "bee");
List<String> result = words.stream()
    .filter(s -> s.length() == 3)
    .collect(Collectors.toList());

// [hey, bee]
```

---

## Mapping Operations

**Q31. Convert all strings to uppercase using map.**

```
List<String> names = Arrays.asList("alice", "bob", "charlie");
List<String> result = names.stream()
    .map(s -> s.toUpperCase())
    .collect(Collectors.toList());

// [ALICE, BOB, CHARLIE]
```

---

**Q32. Convert all strings to lowercase using map.**

```
List<String> names = Arrays.asList("ALICE", "BOB", "CHARLIE");
List<String> result = names.stream()
    .map(s -> s.toLowerCase())
    .collect(Collectors.toList());

// [alice, bob, charlie]
```

---

**Q33. Extract the length of each string in a list.**

```
List<String> words = Arrays.asList("apple", "hi", "stream");
List<Integer> lengths = words.stream()
    .map(s -> s.length())
    .collect(Collectors.toList());

// [5, 2, 6]
```

---

**Q34. Square all numbers in a list.**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
List<Integer> squares = numbers.stream()
    .map(n -> n * n)
    .collect(Collectors.toList());

// [1, 4, 9, 16, 25]
```

---

**Q35. Multiply all numbers by a specific value.**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
List<Integer> result = numbers.stream()
    .map(n -> n * 3)
    .collect(Collectors.toList());

// [3, 6, 9, 12, 15]
```

---

**Q36. Extract specific fields from a list of objects (e.g., get all employee names).**

```
List<Employee> employees = Arrays.asList(
    new Employee("Alice", "IT", 60000),
    new Employee("Bob", "HR", 50000)
);
List<String> names = employees.stream()
    .map(e -> e.getName())
    .collect(Collectors.toList());

// [Alice, Bob]
```

---

**Q37. Convert a list of strings to a list of their lengths.**

```
List<String> words = Arrays.asList("apple", "mango", "banana");
List<Integer> lengths = words.stream()
    .map(String::length)
    .collect(Collectors.toList());

// [5, 5, 6]
```

---

**Q38. Convert a list of integers to a list of their squares.**

```
List<Integer> numbers = Arrays.asList(2, 3, 4);
List<Integer> squares = numbers.stream()
    .map(n -> n * n)
    .collect(Collectors.toList());

// [4, 9, 16]
```

---

**Q39. Add a prefix to all strings in a list.**

```
List<String> names = Arrays.asList("Alice", "Bob");
List<String> result = names.stream()
    .map(s -> "Mr. " + s)
    .collect(Collectors.toList());

// [Mr. Alice, Mr. Bob]
```

---

**Q40. Add a suffix to all strings in a list.**

```
List<String> names = Arrays.asList("Alice", "Bob");
List<String> result = names.stream()
    .map(s -> s + " Jr.")
    .collect(Collectors.toList());

// [Alice Jr., Bob Jr.]
```

---

#### Q41. Extract first character from each string.

```
List<String> words = Arrays.asList("apple", "banana", "cherry");
List<Character> result = words.stream()
    .map(s -> s.charAt(0))
    .collect(Collectors.toList());

// [a, b, c]
```

---

#### Q42. Extract last character from each string.

```
List<String> words = Arrays.asList("apple", "banana", "cherry");
List<Character> result = words.stream()
    .map(s -> s.charAt(s.length() - 1))
    .collect(Collectors.toList());

// [e, a, y]
```

---

#### Q43. Map integers to their corresponding strings.

```
List<Integer> numbers = Arrays.asList(1, 2, 3);
List<String> result = numbers.stream()
    .map(n -> String.valueOf(n))
    .collect(Collectors.toList());

// ["1", "2", "3"]
```

---

#### Q44. Map objects to a different type (e.g., Employee to EmployeeDTO).

```
List<Employee> employees = Arrays.asList(new Employee("Alice", "IT", 60000));
List<EmployeeDTO> dtos = employees.stream()
    .map(e -> new EmployeeDTO(e.getName(), e.getDept()))
    .collect(Collectors.toList());
```

---

#### Q45. Convert a list of strings to integers.

```
List<String> strNums = Arrays.asList("1", "2", "3", "4");
List<Integer> nums = strNums.stream()
    .map(s -> Integer.parseInt(s))
    .collect(Collectors.toList());

// [1, 2, 3, 4]
```

---

## 3. Sorting

---

### Basic Sorting

#### Q46. Sort a list of integers in ascending order.

```
List<Integer> numbers = Arrays.asList(5, 2, 8, 1, 9);
List<Integer> sorted = numbers.stream()
    .sorted()
    .collect(Collectors.toList());

// [1, 2, 5, 8, 9]
```

---

**Q47. Sort a list of integers in descending order.**

```
List<Integer> numbers = Arrays.asList(5, 2, 8, 1, 9);
List<Integer> sorted = numbers.stream()
    .sorted((a, b) -> b - a)
    .collect(Collectors.toList());

// [9, 8, 5, 2, 1]
```

---

**Q48. Sort a list of strings in alphabetical order.**

```
List<String> names = Arrays.asList("Charlie", "Alice", "Bob");
List<String> sorted = names.stream()
    .sorted()
    .collect(Collectors.toList());

// [Alice, Bob, Charlie]
```

---

**Q49. Sort a list of strings in reverse alphabetical order.**

```
List<String> names = Arrays.asList("Charlie", "Alice", "Bob");
List<String> sorted = names.stream()
    .sorted((a, b) -> b.compareTo(a))
    .collect(Collectors.toList());

// [Charlie, Bob, Alice]
```

---

**Q50. Sort a list of strings by their length (ascending).**

```
List<String> words = Arrays.asList("banana", "hi", "apple", "stream");
List<String> sorted = words.stream()
    .sorted((a, b) -> a.length() - b.length())
    .collect(Collectors.toList());

// [hi, apple, banana, stream]
```

---

**Q51. Sort a list of strings by their length (descending).**

```
List<String> words = Arrays.asList("banana", "hi", "apple", "stream");
List<String> sorted = words.stream()
    .sorted((a, b) -> b.length() - a.length())
    .collect(Collectors.toList());

// [banana, stream, apple, hi]
```

---

**Q52. Sort a list ignoring case sensitivity.**



```
List<String> names = Arrays.asList("charlie", "Alice", "bob");
List<String> sorted = names.stream()
    .sorted((a, b) -> a.compareToIgnoreCase(b))
    .collect(Collectors.toList());

// [Alice, bob, charlie]
```

---

**Q53. Sort a list with custom comparator.**

```
List<String> words = Arrays.asList("banana", "apple", "cherry");
List<String> sorted = words.stream()
    .sorted(Comparator.comparing(s -> s.charAt(s.length() - 1)))
    .collect(Collectors.toList());

// Sorted by last character
```

---

## Object Sorting

**Q54. Sort a list of Employee objects by salary in ascending order.**

```
List<Employee> sorted = employees.stream()
    .sorted((a, b) -> Double.compare(a.getSalary(), b.getSalary()))
    .collect(Collectors.toList());
```

---

**Q55. Sort a list of Employee objects by salary in descending order.**

```
List<Employee> sorted = employees.stream()
    .sorted((a, b) -> Double.compare(b.getSalary(), a.getSalary()))
    .collect(Collectors.toList());
```

---

**Q56. Sort a list of Employee objects by name.**

```
List<Employee> sorted = employees.stream()
    .sorted((a, b) -> a.getName().compareTo(b.getName()))
    .collect(Collectors.toList());
```

---

**Q57. Sort a list of Employee objects by age.**

```
List<Employee> sorted = employees.stream()
    .sorted((a, b) -> a.getAge() - b.getAge())
    .collect(Collectors.toList());
```

---

**Q58. Sort a list of Employee objects by multiple fields (e.g., department then salary).**

```
List<Employee> sorted = employees.stream()
    .sorted(Comparator.comparing(Employee::getDept)
        .thenComparing(Employee::getSalary))
    .collect(Collectors.toList());
```

---

**Q59. Sort a list of Employee objects by joining date.**

```
List<Employee> sorted = employees.stream()
    .sorted((a, b) -> a.getJoiningDate().compareTo(b.getJoiningDate()))
    .collect(Collectors.toList());
```

---

**Q60. Sort and then reverse the order.**

```
List<Employee> sorted = employees.stream()
    .sorted(Comparator.comparing(Employee::getName).reversed())
    .collect(Collectors.toList());
```

---

## 4. Reduction & Aggregation

---

### Sum, Min, Max, Average

**Q61. Find the sum of all integers in a list.**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
int sum = numbers.stream()
    .mapToInt(n -> n)
    .sum();
System.out.println(sum); // 15
```

---

**Q62. Find the sum of all even numbers in a list.**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);
int sum = numbers.stream()
    .filter(n -> n % 2 == 0)
    .mapToInt(n -> n)
    .sum();
System.out.println(sum); // 12
```

---

**Q63. Find the sum of all odd numbers in a list.**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);
int sum = numbers.stream()
    .filter(n -> n % 2 != 0)
    .mapToInt(n -> n)
    .sum();
System.out.println(sum); // 9
```

---

**Q64. Find the product of all numbers in a list.**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
int product = numbers.stream()
    .reduce(1, (a, b) -> a * b);
System.out.println(product); // 120
```

---

**Q65. Find the minimum number in a list.**

```
List<Integer> numbers = Arrays.asList(5, 2, 8, 1, 9);
int min = numbers.stream()
    .mapToInt(n -> n)
    .min()
    .getAsInt();
System.out.println(min); // 1
```

---

**Q66. Find the maximum number in a list.**

```
List<Integer> numbers = Arrays.asList(5, 2, 8, 1, 9);
int max = numbers.stream()
    .mapToInt(n -> n)
    .max()
    .getAsInt();
System.out.println(max); // 9
```

---

**Q67. Find the average of all numbers in a list.**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
double avg = numbers.stream()
    .mapToInt(n -> n)
    .average()
    .getAsDouble();
System.out.println(avg); // 3.0
```

---

**Q68. Find the sum of squares of all numbers.**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4);
int sumOfSquares = numbers.stream()
    .mapToInt(n -> n * n)
    .sum();
System.out.println(sumOfSquares); // 30
```

---

**Q69. Find the sum of squares of all even numbers.**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);
int result = numbers.stream()
    .filter(n -> n % 2 == 0)
    .mapToInt(n -> n * n)
    .sum();
System.out.println(result); // 56 (4+16+36)
```

---

**Q70. Calculate the total salary of all employees.**

```
double totalSalary = employees.stream()
    .mapToDouble(e -> e.getSalary())
    .sum();
System.out.println(totalSalary);
```

---

## Reduce Operations

**Q71. Concatenate all strings in a list using reduce.**

```
List<String> words = Arrays.asList("Hello", " ", "World");
String result = words.stream()
    .reduce("", (a, b) -> a + b);
System.out.println(result); // "Hello World"
```

---

**Q72. Find the longest string using reduce.**

```
List<String> words = Arrays.asList("apple", "banana", "kiwi", "strawberry");
String longest = words.stream()
    .reduce("", (a, b) -> a.length() >= b.length() ? a : b);
System.out.println(longest); // strawberry
```

---

**Q73. Find the shortest string using reduce.**

```
List<String> words = Arrays.asList("apple", "banana", "kiwi", "strawberry");
String shortest = words.stream()
    .reduce(words.get(0), (a, b) -> a.length() <= b.length() ? a : b);
System.out.println(shortest); // kiwi
```

---

**Q74. Multiply all numbers in a list using reduce.**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
int product = numbers.stream()
    .reduce(1, (a, b) -> a * b);
System.out.println(product); // 120
```

---

**Q75. Find the maximum number using reduce.**

```
List<Integer> numbers = Arrays.asList(3, 7, 2, 9, 4);
int max = numbers.stream()
    .reduce(Integer.MIN_VALUE, (a, b) -> a > b ? a : b);
System.out.println(max); // 9
```

---

**Q76. Find the minimum number using reduce.**

```
List<Integer> numbers = Arrays.asList(3, 7, 2, 9, 4);
int min = numbers.stream()
    .reduce(Integer.MAX_VALUE, (a, b) -> a < b ? a : b);
System.out.println(min); // 2
```

---

**Q77. Join all strings with a delimiter using reduce.**

```
List<String> words = Arrays.asList("Java", "Stream", "API");
String result = words.stream()
    .reduce((a, b) -> a + ", " + b)
    .orElse("");
System.out.println(result); // Java, Stream, API
```

---

**Q78. Calculate factorial using reduce.**

```
int n = 5;
int factorial = IntStream.rangeClosed(1, n)
    .reduce(1, (a, b) -> a * b);
System.out.println(factorial); // 120
```

---

**Q79. Implement custom reduction logic.**

```
// Sum of squares of even numbers
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
int result = numbers.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * n)
    .reduce(0, (a, b) -> a + b);
System.out.println(result); // 20
```

---

**Q80. Reduce with initial value vs without initial value.**

```
List<Integer> numbers = Arrays.asList(1, 2, 3);

// With identity (always returns int)
int withIdentity = numbers.stream().reduce(0, (a, b) -> a + b);

// Without identity (returns Optional)
Optional<Integer> withoutIdentity = numbers.stream().reduce((a, b) -> a + b);

System.out.println(withIdentity); // 6
System.out.println(withoutIdentity.get()); // 6
```

---

## 5. Collectors — Basic

---

### toList, toSet, toMap

**Q81. Collect stream elements to a List.**

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
List<String> result = names.stream()
    .filter(s -> s.length() > 3)
    .collect(Collectors.toList());

// [Alice, Charlie]
```

---

**Q82. Collect stream elements to a Set.**

```
List<Integer> numbers = Arrays.asList(1, 2, 2, 3, 3, 4);
Set<Integer> result = numbers.stream()
    .collect(Collectors.toSet());

// {1, 2, 3, 4}
```

---

**Q83. Collect stream elements to a specific collection (e.g., TreeSet, LinkedList).**

```
List<String> names = Arrays.asList("Charlie", "Alice", "Bob");

// TreeSet (sorted)
TreeSet<String> treeSet = names.stream()
    .collect(Collectors.toCollection(TreeSet::new));

// LinkedList
LinkedList<String> linkedList = names.stream()
    .collect(Collectors.toCollection(LinkedList::new));
```

---

**Q84. Convert a list to a Map with index as key.**

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
Map<Integer, String> map = IntStream.range(0, names.size())
    .boxed()
    .collect(Collectors.toMap(i -> i, i -> names.get(i)));

// {0=Alice, 1=Bob, 2=Charlie}
```

---

**Q85. Convert a list of objects to a Map (e.g., Employee ID as key, Employee as value).**

```
Map<Integer, Employee> empMap = employees.stream()
    .collect(Collectors.toMap(e -> e.getId(), e -> e));
```

---

**Q86. Convert a list to a Map handling duplicate keys.**

```
List<String> names = Arrays.asList("Alice", "Bob", "Alice");
Map<String, Integer> nameLength = names.stream()
    .collect(Collectors.toMap(
        s -> s,
        s -> s.length(),
        (existing, replacement) -> existing // keep first
    ));
```

---

**Q87. Collect to an unmodifiable list.**

```
List<String> names = Arrays.asList("Alice", "Bob");
List<String> unmodifiable = names.stream()
    .collect(Collectors.toUnmodifiableList());
```

---

**Q88. Collect to an unmodifiable set.**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 2);
Set<Integer> unmodifiable = numbers.stream()
    .collect(Collectors.toUnmodifiableSet());
```

---

**Q89. Collect to an unmodifiable map.**

```
Map<String, Integer> unmodifiable = employees.stream()
    .collect(Collectors.toUnmodifiableMap(
        e -> e.getName(),
        e -> e.getSalary()
    ));
```

---

**Q90. Convert stream to array.**

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
String[] arr = names.stream().toArray(String[]::new);
```

---

## Joining Operations

**Q91. Join all strings in a list with comma separator.**

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
String result = names.stream()
    .collect(Collectors.joining(", "));
System.out.println(result); // Alice, Bob, Charlie
```

**Q92. Join all strings with a custom delimiter.**

```
List<String> words = Arrays.asList("Java", "Stream", "API");
String result = words.stream()
    .collect(Collectors.joining(" | "));
System.out.println(result); // Java | Stream | API
```

**Q93. Join strings with prefix and suffix.**

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
String result = names.stream()
    .collect(Collectors.joining(", ", "[", "]"));
System.out.println(result); // [Alice, Bob, Charlie]
```

**Q94. Join strings with space separator.**

```
List<String> words = Arrays.asList("Hello", "World", "Java");
String result = words.stream()
    .collect(Collectors.joining(" "));
System.out.println(result); // Hello World Java
```

**Q95. Concatenate all strings without separator.**

```
List<String> parts = Arrays.asList("Java", "Stream", "API");
String result = parts.stream()
    .collect(Collectors.joining());
System.out.println(result); // JavaStreamAPI
```

---

## 6. Collectors – GroupingBy

### Basic Grouping

**Q96. Group strings by their length.**

```
List<String> words = Arrays.asList("hi", "hello", "hey", "stream", "java");
Map<Integer, List<String>> grouped = words.stream()
    .collect(Collectors.groupingBy(s -> s.length()));
// {2=[hi], 3=[hey], 4=[java], 5=[hello], 6=[stream]}
```

---

#### Q97. Group numbers by even/odd.

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);
Map<String, List<Integer>> grouped = numbers.stream()
    .collect(Collectors.groupingBy(
        n -> n % 2 == 0 ? "Even" : "Odd"
    ));
// {Even=[2, 4, 6], Odd=[1, 3, 5]}
```

---

#### Q98. Group employees by department.

```
Map<String, List<Employee>> grouped = employees.stream()
    .collect(Collectors.groupingBy(e -> e.getDept()));
```

---

#### Q99. Group employees by gender.

```
Map<String, List<Employee>> grouped = employees.stream()
    .collect(Collectors.groupingBy(e -> e.getGender()));
```

---

#### Q100. Group employees by age.

```
Map<Integer, List<Employee>> grouped = employees.stream()
    .collect(Collectors.groupingBy(e -> e.getAge()));
```

---

## 6. Collectors — GroupingB

### Basic Grouping (*continued*)

#### Q101. Group employees by salary range.

```
Map<String, List<Employee>> grouped = employees.stream()
    .collect(Collectors.groupingBy(e -> {
        if (e.getSalary() < 30000) return "Low";
        else if (e.getSalary() < 70000) return "Mid";
        else return "High";
    }));
```

---

#### Q102. Group strings by their first character.

```
List<String> words = Arrays.asList("apple", "banana", "avocado", "blueberry", "cherry");
Map<Character, List<String>> grouped = words.stream()
    .collect(Collectors.groupingBy(s -> s.charAt(0)));
// {a=[apple, avocado], b=[banana, blueberry], c=[cherry]}
```

---

#### Q103. Group numbers by their remainder when divided by a number.



```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9);
Map<Integer, List<Integer>> grouped = numbers.stream()
    .collect(Collectors.groupingBy(n -> n % 3));

// {0=[3, 6, 9], 1=[1, 4, 7], 2=[2, 5, 8]}
```

---

**Q104. Group employees by city.**

```
Map<String, List<Employee>> grouped = employees.stream()
    .collect(Collectors.groupingBy(e -> e.getCity()));
```

---

**Q105. Group employees by joining year.**

```
Map<Integer, List<Employee>> grouped = employees.stream()
    .collect(Collectors.groupingBy(
        e -> e.getJoiningDate().getYear()
    ));
```

---

## Counting with GroupingBy

**Q106. Count employees in each department.**

```
Map<String, Long> countByDept = employees.stream()
    .collect(Collectors.groupingBy(
        e -> e.getDept(),
        Collectors.counting()
    ));
```

---

**Q107. Count strings of each length.**

```
List<String> words = Arrays.asList("hi", "hey", "hello", "java", "bee");
Map<Integer, Long> countByLength = words.stream()
    .collect(Collectors.groupingBy(
        s -> s.length(),
        Collectors.counting()
    ));
```

---

**Q108. Count even and odd numbers.**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);
Map<String, Long> count = numbers.stream()
    .collect(Collectors.groupingBy(
        n -> n % 2 == 0 ? "Even" : "Odd",
        Collectors.counting()
    ));

// {Even=3, Odd=3}
```

---

**Q109. Count male and female employees.**

```
Map<String, Long> count = employees.stream()
    .collect(Collectors.groupingBy(
        e -> e.getGender(),
        Collectors.counting()
    ));
```

---

**Q110. Find department with maximum number of employees.**

```
Map<String, Long> countByDept = employees.stream()
    .collect(Collectors.groupingBy(e -> e.getDept(), Collectors.counting()));

String maxDept = countByDept.entrySet().stream()
    .max((a, b) -> Long.compare(a.getValue(), b.getValue()))
    .map(e -> e.getKey())
    .orElse("None");

System.out.println(maxDept);
```

---

**Q111. Find the frequency of each character in a string.**

```
String str = "programming";
Map<Character, Long> freq = str.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(c -> c, Collectors.counting()));
```

---

**Q112. Count occurrences of each element in a list.**

```
List<String> names = Arrays.asList("Alice", "Bob", "Alice", "Charlie", "Bob", "Alice");
Map<String, Long> freq = names.stream()
    .collect(Collectors.groupingBy(s -> s, Collectors.counting()));

// {Alice=3, Bob=2, Charlie=1}
```

---

**Q113. Find the most frequent element in a list.**

```
List<String> names = Arrays.asList("Alice", "Bob", "Alice", "Charlie", "Bob", "Alice");
String mostFrequent = names.stream()
    .collect(Collectors.groupingBy(s -> s, Collectors.counting()))
    .entrySet().stream()
    .max((a, b) -> Long.compare(a.getValue(), b.getValue()))
    .map(e -> e.getKey())
    .orElse("None");

System.out.println(mostFrequent); // Alice
```

---

## Downstream Collectors with GroupingBy

**Q114. Group employees by department and collect their names.**

```
Map<String, List<String>> namesByDept = employees.stream()
    .collect(Collectors.groupingBy(
        e -> e.getDept(),
        Collectors.mapping(e -> e.getName(), Collectors.toList())
    ));
```

---

**Q115. Group employees by department and find average salary.**

```
Map<String, Double> avgSalaryByDept = employees.stream()
    .collect(Collectors.groupingBy(
        e -> e.getDept(),
        Collectors.averagingDouble(e -> e.getSalary())
    ));
```

---

**Q116. Group employees by department and find total salary.**

```
Map<String, Double> totalSalaryByDept = employees.stream()
    .collect(Collectors.groupingBy(
        e -> e.getDept(),
        Collectors.summingDouble(e -> e.getSalary())
    ));
```

---

**Q117. Group employees by department and find maximum salary.**

```
Map<String, Optional<Employee>> maxSalaryByDept = employees.stream()
    .collect(Collectors.groupingBy(
        e -> e.getDept(),
        Collectors.maxBy((a, b) -> Double.compare(a.getSalary(), b.getSalary()))
    ));
```

---

**Q118. Group employees by department and find minimum salary.**

```
Map<String, Optional<Employee>> minSalaryByDept = employees.stream()
    .collect(Collectors.groupingBy(
        e -> e.getDept(),
        Collectors.minBy((a, b) -> Double.compare(a.getSalary(), b.getSalary()))
    ));
```

---

**Q119. Group employees by department and count them.**

```
Map<String, Long> countByDept = employees.stream()
    .collect(Collectors.groupingBy(e -> e.getDept(), Collectors.counting()));
```

---

**Q120. Group employees by department and find the highest paid employee.**

```
Map<String, Optional<Employee>> highestPaidByDept = employees.stream()
    .collect(Collectors.groupingBy(
        e -> e.getDept(),
        Collectors.maxBy(Comparator.comparingDouble(Employee::getSalary))
    ));
```

---

**Q121. Group employees by department and sort by salary.**

```
Map<String, List<Employee>> sortedByDept = employees.stream()
    .collect(Collectors.groupingBy(
        e -> e.getDept(),
        Collectors.collectingAndThen(
            Collectors.toList(),
            list -> list.stream()
                .sorted((a, b) -> Double.compare(a.getSalary(), b.getSalary()))
                .collect(Collectors.toList())
        )
    ));
```

---

**Q122. Group students by grade and find average marks.**

```
Map<String, Double> avgMarksByGrade = students.stream()
    .collect(Collectors.groupingBy(
        s -> s.getGrade(),
        Collectors.averagingDouble(s -> s.getMarks())
    ));
```

---

**Q123. Group employees by department and collect to Set.**

```
Map<String, Set<Employee>> setByDept = employees.stream()
    .collect(Collectors.groupingBy(e -> e.getDept(), Collectors.toSet()));
```

---

**Q124. Group by multiple levels (nested grouping).**

```
// Group by department, then by gender
Map<String, Map<String, List<Employee>>> nested = employees.stream()
    .collect(Collectors.groupingBy(
        e -> e.getDept(),
        Collectors.groupingBy(e -> e.getGender())
    ));
```

---

## 7. Collectors — Advanced

### PartitioningBy

**Q125. Partition numbers into even and odd using partitioningBy.**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);
Map<Boolean, List<Integer>> partitioned = numbers.stream()
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));
// {true=[2, 4, 6], false=[1, 3, 5]}
```

---

**Q126. Partition employees into high earners (>50000) and others.**

```
Map<Boolean, List<Employee>> partitioned = employees.stream()
    .collect(Collectors.partitioningBy(e -> e.getSalary() > 50000));
// true = high earners, false = others
```

---

**Q127. Partition strings into palindromes and non-palindromes.**

```
List<String> words = Arrays.asList("madam", "java", "level", "stream", "racecar");
Map<Boolean, List<String>> partitioned = words.stream()
    .collect(Collectors.partitioningBy(
        s -> s.equals(new StringBuilder(s).reverse().toString())
    ));
```

---

**Q128. Partition employees by age (above 30 and below 30).**

```
Map<Boolean, List<Employee>> partitioned = employees.stream()
    .collect(Collectors.partitioningBy(e -> e.getAge() > 30));
```

---

**Q129. Partition products by availability (in stock or out of stock).**

```
Map<Boolean, List<Product>> partitioned = products.stream()
    .collect(Collectors.partitioningBy(p -> p.getStock() > 0));
```

---

## Mapping Collector

**Q130. Group employees by department and map to their names.**

```
Map<String, List<String>> namesByDept = employees.stream()
    .collect(Collectors.groupingBy(
        e -> e.getDept(),
        Collectors.mapping(Employee::getName, Collectors.toList())
    ));
```

---

**Q131. Group employees by department and map to their salaries.**

```
Map<String, List<Double>> salariesByDept = employees.stream()
    .collect(Collectors.groupingBy(
        e -> e.getDept(),
        Collectors.mapping(Employee::getSalary, Collectors.toList())
    ));
```

---

**Q132. Group strings by length and convert to uppercase.**

```
Map<Integer, List<String>> result = words.stream()
    .collect(Collectors.groupingBy(
        String::length,
        Collectors.mapping(String::toUpperCase, Collectors.toList())
    ));
```

---

**Q133. Use mapping collector to transform data after grouping.**

```
// Group by dept, get CSV of names
Map<String, String> csvNamesByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDept,
        Collectors.mapping(Employee::getName, Collectors.joining(", "))
    ));
```

---

## FlatMapping Collector

**Q134. Group employees by department and flatten their skills.**

```
// Assuming Employee has List<String> getSkills()
Map<String, List<String>> skillsByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDept,
        Collectors.flatMapping(e -> e.getSkills().stream(), Collectors.toList())
    ));
```

---

**Q135. Flatten nested lists after grouping.**

```
Map<String, List<String>> result = data.stream()
    .collect(Collectors.groupingBy(
        Item::getCategory,
        Collectors.flatMapping(item -> item.getTags().stream(), Collectors.toList())
    ));
```

---

**Q136. Group orders by customer and flatten line items.**

```
Map<String, List<String>> itemsByCustomer = orders.stream()
    .collect(Collectors.groupingBy(
        Order::getCustomer,
        Collectors.flatMapping(o -> o.getItems().stream(), Collectors.toList())
    ));
```

---

**Q137. Use flatMapping with groupingBy.**

```
Map<String, Set<String>> result = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDept,
        Collectors.flatMapping(e -> e.getSkills().stream(), Collectors.toSet())
    ));
```

---

## CollectingAndThen

**Q138. Group employees by department and get unmodifiable lists.**

```
Map<String, List<Employee>> result = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDept,
        Collectors.collectingAndThen(Collectors.toList(), Collections::unmodifiableList)
    ));
```

---

**Q139. Find max element and wrap in custom object using collectingAndThen.**

```
String longestName = employees.stream()
    .collect(Collectors.collectingAndThen(
        Collectors.maxBy(Comparator.comparingInt(e -> e.getName().length())),
        opt -> opt.map(Employee::getName).orElse("N/A")
    ));
```

---

**Q140. Calculate average and format result using collectingAndThen.**

```
String formatted = employees.stream()
    .collect(Collectors.collectingAndThen(
        Collectors.averagingDouble(Employee::getSalary),
        avg -> String.format("Average Salary: %.2f", avg)
    ));
```

---

**Q141. Sort after collecting using collectingAndThen.**

```
List<Employee> sortedList = employees.stream()
    .collect(Collectors.collectingAndThen(
        Collectors.toList(),
        list -> { list.sort(Comparator.comparing(Employee::getName)); return list; }
    ));
```

---

**Q142. Find second highest salary in each department.**

```
Map<String, Optional<Double>> result = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDept,
        Collectors.collectingAndThen(
            Collectors.mapping(Employee::getSalary,
                Collectors.collectingAndThen(
                    Collectors.toList(),
                    list -> list.stream().distinct().sorted(Collections.reverseOrder()).skip(1).findFirst()
                )
            ),
            opt -> opt
        )
    ));
```

---

## Filtering Collector

**Q143. Group employees by department and filter those earning >50000.**

```
Map<String, List<Employee>> result = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDept,
        Collectors.filtering(e -> e.getSalary() > 50000, Collectors.toList())
    ));
```

---

**Q144. Use filtering collector with groupingBy to exclude elements.**

```
Map<String, Long> countHighEarners = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDept,
        Collectors.filtering(e -> e.getSalary() > 60000, Collectors.counting())
    ));
```

---

**Q145. Filter and group in a single operation.**

```
// Get names of employees earning > 50000 grouped by dept
Map<String, List<String>> result = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDept,
        Collectors.filtering(
            e -> e.getSalary() > 50000,
            Collectors.mapping(Employee::getName, Collectors.toList())
        )
    ));
```

---

## 8. FlatMap Operations

---

### Basic FlatMap

**Q146. Flatten a list of lists into a single list.**

```
List<List<Integer>> nested = Arrays.asList(
    Arrays.asList(1, 2, 3),
    Arrays.asList(4, 5),
    Arrays.asList(6, 7, 8)
);
List<Integer> flat = nested.stream()
    .flatMap(list -> list.stream())
    .collect(Collectors.toList());
// [1, 2, 3, 4, 5, 6, 7, 8]
```

---

**Q147. Flatten a list of arrays into a single list.**

```
List<String[]> arrays = Arrays.asList(
    new String[]{"a", "b"},
    new String[]{"c", "d"}
);
List<String> flat = arrays.stream()
    .flatMap(arr -> Arrays.stream(arr))
    .collect(Collectors.toList());
// [a, b, c, d]
```

---

**Q148. Convert a list of strings to a stream of characters.**

```
List<String> words = Arrays.asList("Java", "Stream");
List<Character> chars = words.stream()
    .flatMap(s -> s.chars().mapToObj(c -> (char) c))
    .collect(Collectors.toList());
// [J, a, v, a, S, t, r, e, a, m]
```

---

**Q149. Split strings and flatten the results.**

```
List<String> sentences = Arrays.asList("hello world", "java stream api");
List<String> words = sentences.stream()
    .flatMap(s -> Arrays.stream(s.split(" ")))
    .collect(Collectors.toList());
// [hello, world, java, stream, api]
```

---

**Q150. Flatten a list of sets into a single list.**



```
List<Set<String>> sets = Arrays.asList(
    new HashSet<>(Arrays.asList("a", "b")),
    new HashSet<>(Arrays.asList("c", "d"))
);
List<String> flat = sets.stream()
    .flatMap(set -> set.stream())
    .collect(Collectors.toList());
```

---

#### Q151. Flatten Optional values.

```
List<Optional<String>> optionals = Arrays.asList(
    Optional.of("apple"),
    Optional.empty(),
    Optional.of("banana")
);
List<String> result = optionals.stream()
    .flatMap(opt -> opt.stream())
    .collect(Collectors.toList());

// [apple, banana]
```

---

#### Q152. Flatten nested collections.

```
List<List<List<Integer>>> nested = Arrays.asList(
    Arrays.asList(Arrays.asList(1, 2), Arrays.asList(3, 4)),
    Arrays.asList(Arrays.asList(5, 6))
);
List<Integer> flat = nested.stream()
    .flatMap(l -> l.stream())
    .flatMap(l -> l.stream())
    .collect(Collectors.toList());

// [1, 2, 3, 4, 5, 6]
```

---

## Complex FlatMap Scenarios

#### Q153. Flatten a list of Employee lists (from multiple departments).

```
List<List<Employee>> departments = Arrays.asList(itDept, hrDept, financeDept);
List<Employee> allEmployees = departments.stream()
    .flatMap(dept -> dept.stream())
    .collect(Collectors.toList());
```

---

#### Q154. Split sentences into words and flatten.

```
List<String> sentences = Arrays.asList("I love Java", "Stream API is powerful");
List<String> words = sentences.stream()
    .flatMap(s -> Arrays.stream(s.split(" ")))
    .collect(Collectors.toList());
```

---

#### Q155. Extract all unique characters from a list of strings.

```
List<String> words = Arrays.asList("hello", "world", "java");
Set<Character> uniqueChars = words.stream()
    .flatMap(s -> s.chars().mapToObj(c -> (char) c))
    .collect(Collectors.toSet());
```

---

**Q156. Flatten a list of lists and filter results.**

```
List<List<Integer>> nested = Arrays.asList(
    Arrays.asList(1, 2, 3, 4),
    Arrays.asList(5, 6, 7, 8)
);
List<Integer> evenNumbers = nested.stream()
    .flatMap(list -> list.stream())
    .filter(n -> n % 2 == 0)
    .collect(Collectors.toList());

// [2, 4, 6, 8]
```

---

**Q157. Flatten and map in combination.**

```
List<String> words = Arrays.asList("hello", "world");
List<String> upperChars = words.stream()
    .flatMap(s -> s.chars().mapToObj(c -> String.valueOf((char) c)))
    .map(String::toUpperCase)
    .collect(Collectors.toList());
```

---

**Q158. Flatten a stream of streams.**

```
Stream<Stream<Integer>> streamOfStreams = Stream.of(
    Stream.of(1, 2, 3),
    Stream.of(4, 5, 6)
);
List<Integer> flat = streamOfStreams.flatMap(s -> s)
    .collect(Collectors.toList());

// [1, 2, 3, 4, 5, 6]
```

---

**Q159. Get all unique words from a list of sentences.**

```
List<String> sentences = Arrays.asList("I love Java", "Java is great", "I love streams");
Set<String> uniqueWords = sentences.stream()
    .flatMap(s -> Arrays.stream(s.split(" ")))
    .collect(Collectors.toSet());
```

---

**Q160. Flatten map values into a single list.**

```
Map<String, List<String>> map = Map.of(
    "fruits", Arrays.asList("apple", "banana"),
    "veggies", Arrays.asList("carrot", "pea")
);
List<String> allValues = map.values().stream()
    .flatMap(list -> list.stream())
    .collect(Collectors.toList());
```

---

## 9. String Manipulation

### Character Operations

**Q161. Find the first non-repeated character in a string.**

```
String str = "programming";
char result = str.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(c -> c, LinkedHashMap::new, Collectors.counting()))
    .entrySet().stream()
    .filter(e -> e.getValue() == 1)
    .map(e -> e.getKey())
    .findFirst()
    .orElse('\0');
System.out.println(result); // 'p'... wait: actually 'o' – first non-repeated
```

---

**Q162. Find all duplicate characters in a string.**

```
String str = "programming";
Set<Character> duplicates = str.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(c -> c, Collectors.counting()))
    .entrySet().stream()
    .filter(e -> e.getValue() > 1)
    .map(e -> e.getKey())
    .collect(Collectors.toSet());
// [r, g, m]
```

---

**Q163. Count the frequency of each character in a string.**

```
String str = "hello";
Map<Character, Long> freq = str.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(c -> c, Collectors.counting()));
// {h=1, e=1, l=2, o=1}
```

---

**Q164. Find the most frequent character in a string.**

```
String str = "programming";
char mostFrequent = str.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(c -> c, Collectors.counting()))
    .entrySet().stream()
    .max((a, b) -> Long.compare(a.getValue(), b.getValue()))
    .map(e -> e.getKey())
    .orElse('\0');
```

---

**Q165. Remove duplicate characters from a string.**

```
String str = "programming";
String result = str.chars()
    .distinct()
    .collect(StringBuilder::new, StringBuilder::appendCodePoint, StringBuilder::append)
    .toString();
System.out.println(result); // "progamin"
```

---

**Q166. Check if two strings are anagrams.**

```
String s1 = "listen";
String s2 = "silent";
boolean isAnagram = s1.chars().sorted().boxed().collect(Collectors.toList())
                    .equals(s2.chars().sorted().boxed().collect(Collectors.toList()));
System.out.println(isAnagram); // true
```

---

**Q167. Reverse each word in a sentence.**

```
String sentence = "Hello World Java";
String result = Arrays.stream(sentence.split(" "))
    .map(w -> new StringBuilder(w).reverse().toString())
    .collect(Collectors.joining(" "));
System.out.println(result); // olleH dlrow avaJ
```

---

**Q168. Reverse a string using streams.**

```
String str = "Hello";
String reversed = str.chars()
    .mapToObj(c -> String.valueOf((char) c))
    .reduce("", (a, b) -> b + a);
System.out.println(reversed); // olleH
```

---

**Q169. Check if a string is a palindrome using streams.**

```
String str = "racecar";
boolean isPalindrome = IntStream.range(0, str.length() / 2)
    .allMatch(i -> str.charAt(i) == str.charAt(str.length() - 1 - i));
System.out.println(isPalindrome); // true
```

---

## Word Operations

**Q170. Find the longest word in a sentence.**

```
String sentence = "I love Java stream programming";
String longest = Arrays.stream(sentence.split(" "))
    .max(Comparator.comparingInt(String::length))
    .orElse("");
System.out.println(longest); // programming
```

---

**Q171. Find the shortest word in a sentence.**

```
String sentence = "I love Java stream programming";
String shortest = Arrays.stream(sentence.split(" "))
    .min(Comparator.comparingInt(String::length))
    .orElse("");
System.out.println(shortest); // I
```

---

**Q172. Count the number of words in a sentence.**

```
String sentence = "Java Stream API is powerful";
long count = Arrays.stream(sentence.split(" "))
    .filter(w -> !w.isEmpty())
    .count();
System.out.println(count); // 5
```

---

**Q173. Count vowels in a string.**

```
String str = "programming";
long count = str.chars()
    .filter(c -> "aeiouAEIOU".indexOf(c) != -1)
    .count();
System.out.println(count); // 3
```

---

**Q174. Count consonants in a string.**

```
String str = "programming";
long count = str.chars()
    .filter(Character::isLetter)
    .filter(c -> "aeiouAEIOU".indexOf(c) == -1)
    .count();
System.out.println(count); // 8
```

---

**Q175. Find all palindromic words in a sentence.**

```
String sentence = "madam went to the racecar level";
List<String> palindromes = Arrays.stream(sentence.split(" "))
    .filter(w -> w.equals(new StringBuilder(w).reverse().toString()))
    .collect(Collectors.toList());
// [madam, racecar, level]
```

---

**Q176. Capitalize the first letter of each word.**

```
String sentence = "hello world java";
String result = Arrays.stream(sentence.split(" "))
    .map(w -> Character.toUpperCase(w.charAt(0)) + w.substring(1))
    .collect(Collectors.joining(" "));
System.out.println(result); // Hello World Java
```

---

**Q177. Convert string to title case.**

```
String sentence = "the quick brown fox";
String titleCase = Arrays.stream(sentence.split(" "))
    .map(w -> w.isEmpty() ? w : Character.toUpperCase(w.charAt(0)) + w.substring(1).toLowerCase())
    .collect(Collectors.joining(" "));
System.out.println(titleCase); // The Quick Brown Fox
```

---

**Q178. Remove all whitespace from a string.**

```
String str = "Hello World Java Stream";
String result = str.chars()
    .filter(c -> !Character.isWhitespace(c))
    .collect(StringBuilder::new, StringBuilder::appendCodePoint, StringBuilder::append)
    .toString();
System.out.println(result); // HelloWorldJavaStream
```

---

**Q179. Find words with a specific length.**

```
String sentence = "I love Java stream programming";
int targetLength = 4;
List<String> result = Arrays.stream(sentence.split(" "))
    .filter(w -> w.length() == targetLength)
    .collect(Collectors.toList());

// [love, Java]
```

---

**Q180. Find words starting with a vowel.**

```
String sentence = "apple and orange banana";
List<String> result = Arrays.stream(sentence.split(" "))
    .filter(w -> "aeiouAEIOU".indexOf(w.charAt(0)) != -1)
    .collect(Collectors.toList());

// [apple, and, orange]
```

---

## 10. Finding & Matching

---

### Find Operations

**Q181. Find the first element in a stream.**

```
List<Integer> numbers = Arrays.asList(5, 3, 8, 1, 4);
Optional<Integer> first = numbers.stream().findFirst();
System.out.println(first.get()); // 5
```

---

**Q182. Find any element in a parallel stream.**

```
List<Integer> numbers = Arrays.asList(5, 3, 8, 1, 4);
Optional<Integer> any = numbers.parallelStream().findAny();
System.out.println(any.get()); // any element (non-deterministic)
```

---

**Q183. Find the first even number in a list.**

```
List<Integer> numbers = Arrays.asList(1, 3, 5, 4, 6, 8);
Optional<Integer> firstEven = numbers.stream()
    .filter(n -> n % 2 == 0)
    .findFirst();

System.out.println(firstEven.get()); // 4
```

---

**Q184. Find the first string starting with 'A'.**

```
List<String> names = Arrays.asList("Bob", "Alice", "Anna", "Charlie");
Optional<String> first = names.stream()
    .filter(s -> s.startsWith("A"))
    .findFirst();
System.out.println(first.get()); // Alice
```

---

**Q185. Find the first employee with salary > 50000.**

```
Optional<Employee> emp = employees.stream()
    .filter(e -> e.getSalary() > 50000)
    .findFirst();
emp.ifPresent(e -> System.out.println(e.getName()));
```

---

**Q186. Find the first element greater than 100.**

```
List<Integer> numbers = Arrays.asList(50, 80, 120, 200);
Optional<Integer> result = numbers.stream()
    .filter(n -> n > 100)
    .findFirst();
System.out.println(result.get()); // 120
```

---

**Q187. Find the last element in a stream.**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
Optional<Integer> last = numbers.stream()
    .reduce((a, b) -> b);
System.out.println(last.get()); // 5
```

---

**Q188. Find the nth element in a stream.**

```
List<Integer> numbers = Arrays.asList(10, 20, 30, 40, 50);
int n = 3; // 3rd element (1-based)
Optional<Integer> nth = numbers.stream()
    .skip(n - 1)
    .findFirst();
System.out.println(nth.get()); // 30
```

---

## Matching Operations

**Q189. Check if any element matches a condition (anyMatch).**

```
List<Integer> numbers = Arrays.asList(1, 3, 5, 8, 9);
boolean hasEven = numbers.stream().anyMatch(n -> n % 2 == 0);
System.out.println(hasEven); // true
```

---

**Q190. Check if all elements match a condition (allMatch).**

```
List<Integer> numbers = Arrays.asList(2, 4, 6, 8);
boolean allEven = numbers.stream().allMatch(n -> n % 2 == 0);
System.out.println(allEven); // true
```

---

**Q191. Check if no element matches a condition (noneMatch).**

```
List<Integer> numbers = Arrays.asList(1, 3, 5, 7);
boolean noEven = numbers.stream().noneMatch(n -> n % 2 == 0);
System.out.println(noEven); // true
```

---

**Q192. Check if any string contains a specific substring.**

```
List<String> words = Arrays.asList("apple", "banana", "cherry");
boolean result = words.stream().anyMatch(s -> s.contains("an"));
System.out.println(result); // true
```

---

**Q193. Check if all numbers are positive.**

```
List<Integer> numbers = Arrays.asList(3, 5, 7, 9);
boolean allPositive = numbers.stream().allMatch(n -> n > 0);
System.out.println(allPositive); // true
```

---

**Q194. Check if all strings are non-empty.**

```
List<String> words = Arrays.asList("hello", "world", "java");
boolean allNonEmpty = words.stream().allMatch(s -> !s.isEmpty());
System.out.println(allNonEmpty); // true
```

---

**Q195. Check if any employee salary is above 100000.**

```
boolean hasHighEarner = employees.stream()
    .anyMatch(e -> e.getSalary() > 100000);
System.out.println(hasHighEarner);
```

---

**Q196. Check if all employees are adults (age >= 18).**

```
boolean allAdults = employees.stream()
    .allMatch(e -> e.getAge() >= 18);
System.out.println(allAdults);
```

---

**Q197. Check if no employee has zero salary.**

```
boolean noZeroSalary = employees.stream()
    .noneMatch(e -> e.getSalary() == 0);
System.out.println(noZeroSalary);
```

---

**Q198. Check if any number is prime.**

```
List<Integer> numbers = Arrays.asList(4, 6, 8, 7, 10);
boolean hasPrime = numbers.stream()
    .anyMatch(n -> n > 1 && IntStream.rangeClosed(2, (int) Math.sqrt(n)).allMatch(i -> n % i != 0));
System.out.println(hasPrime); // true (7 is prime)
```

---



## 11. Distinct & Unique Elements

---

### Distinct Operations

**Q199. Remove duplicate elements from a list.**

```
List<Integer> numbers = Arrays.asList(1, 2, 2, 3, 3, 4, 1);
List<Integer> distinct = numbers.stream()
    .distinct()
    .collect(Collectors.toList());

// [1, 2, 3, 4]
```

---

**Q200. Get distinct numbers from a list.**

```
List<Integer> numbers = Arrays.asList(5, 3, 5, 2, 8, 3, 1);
List<Integer> distinct = numbers.stream()
    .distinct()
    .sorted()
    .collect(Collectors.toList());

// [1, 2, 3, 5, 8]
```

---

## 11. Distinct & Unique Elements (*continued*)

---

### Distinct Operations

**Q201. Get distinct strings (case-insensitive).**

```
List<String> names = Arrays.asList("Alice", "alice", "BOB", "bob", "Charlie");
List<String> distinct = names.stream()
    .map(String::toLowerCase)
    .distinct()
    .collect(Collectors.toList());

// [alice, bob, charlie]
```

---

**Q202. Find duplicate elements in a list.**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 2, 4, 3, 5);
Set<Integer> seen = new HashSet<>();
List<Integer> duplicates = numbers.stream()
    .filter(n -> !seen.add(n))
    .collect(Collectors.toList());

// [2, 3]
```

---

**Q203. Find all unique elements (appearing only once).**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 2, 4, 3, 5);
Map<Integer, Long> freq = numbers.stream()
    .collect(Collectors.groupingBy(n -> n, Collectors.counting()));

List<Integer> unique = freq.entrySet().stream()
    .filter(e -> e.getValue() == 1)
    .map(e -> e.getKey())
    .collect(Collectors.toList());

// [1, 4, 5]
```

---

#### Q204. Get distinct employee names.

```
List<String> distinctNames = employees.stream()
    .map(Employee::getName)
    .distinct()
    .collect(Collectors.toList());
```

---

#### Q205. Remove duplicate objects based on a property.

```
// Remove duplicate employees based on name
Map<String, Employee> seen = new LinkedHashMap<>();
List<Employee> distinct = employees.stream()
    .filter(e -> seen.putIfAbsent(e.getName(), e) == null)
    .collect(Collectors.toList());
```

---

#### Q206. Find elements appearing more than once.

```
List<String> names = Arrays.asList("Alice", "Bob", "Alice", "Charlie", "Bob");
List<String> moreThanOnce = names.stream()
    .collect(Collectors.groupingBy(s -> s, Collectors.counting()))
    .entrySet().stream()
    .filter(e -> e.getValue() > 1)
    .map(e -> e.getKey())
    .collect(Collectors.toList());
// [Alice, Bob]
```

---

### Set Operations

#### Q207. Find the intersection of two lists.

```
List<Integer> list1 = Arrays.asList(1, 2, 3, 4, 5);
List<Integer> list2 = Arrays.asList(3, 4, 5, 6, 7);
List<Integer> intersection = list1.stream()
    .filter(list2::contains)
    .collect(Collectors.toList());
// [3, 4, 5]
```

---

#### Q208. Find the union of two lists.

```
List<Integer> list1 = Arrays.asList(1, 2, 3, 4);
List<Integer> list2 = Arrays.asList(3, 4, 5, 6);
List<Integer> union = Stream.concat(list1.stream(), list2.stream())
    .distinct()
    .collect(Collectors.toList());
// [1, 2, 3, 4, 5, 6]
```

---

#### Q209. Find elements in list1 but not in list2.

```
List<Integer> list1 = Arrays.asList(1, 2, 3, 4, 5);
List<Integer> list2 = Arrays.asList(3, 4, 5, 6, 7);
List<Integer> diff = list1.stream()
    .filter(n -> !list2.contains(n))
    .collect(Collectors.toList());
// [1, 2]
```

---

**Q210. Find common elements between two lists.**

```
List<String> list1 = Arrays.asList("Alice", "Bob", "Charlie");
List<String> list2 = Arrays.asList("Bob", "David", "Alice");
List<String> common = list1.stream()
    .filter(list2::contains)
    .collect(Collectors.toList());

// [Alice, Bob]
```

---

**Q211. Combine two lists and remove duplicates.**

```
List<Integer> list1 = Arrays.asList(1, 2, 3);
List<Integer> list2 = Arrays.asList(2, 3, 4, 5);
List<Integer> combined = Stream.concat(list1.stream(), list2.stream())
    .distinct()
    .collect(Collectors.toList());

// [1, 2, 3, 4, 5]
```

---

**Q212. Find symmetric difference between two sets.**

```
List<Integer> list1 = Arrays.asList(1, 2, 3, 4, 5);
List<Integer> list2 = Arrays.asList(3, 4, 5, 6, 7);

// Elements in one list but not the other
List<Integer> symDiff = Stream.concat(
    list1.stream().filter(n -> !list2.contains(n)),
    list2.stream().filter(n -> !list1.contains(n))
).collect(Collectors.toList());

// [1, 2, 6, 7]
```

---

## 12. Parallel Streams

---

**Q213. Calculate sum using parallel streams.**

```
List<Integer> numbers = IntStream.rangeClosed(1, 1000).boxed().collect(Collectors.toList());
int sum = numbers.parallelStream()
    .mapToInt(n -> n)
    .sum();
System.out.println(sum); // 500500
```

---

**Q214. Find max element using parallel stream.**

```
List<Integer> numbers = Arrays.asList(3, 7, 1, 9, 4, 6, 8);
Optional<Integer> max = numbers.parallelStream()
    .max(Integer::compareTo);
System.out.println(max.get()); // 9
```

---

**Q215. Filter and collect using parallel stream.**

```
List<Integer> numbers = IntStream.rangeClosed(1, 100).boxed().collect(Collectors.toList());
List<Integer> evens = numbers.parallelStream()
    .filter(n -> n % 2 == 0)
    .collect(Collectors.toList());
```

---

**Q216. Sort a large list using parallel stream.**

```
List<Integer> numbers = IntStream.rangeClosed(1, 10000)
    .boxed()
    .collect(Collectors.toList());

Collections.shuffle(numbers);
List<Integer> sorted = numbers.parallelStream()
    .sorted()
    .collect(Collectors.toList());
```

---

**Q217. Process large dataset in parallel.**

```
// Process 1 million records in parallel
List<Employee> result = largeEmployeeList.parallelStream()
    .filter(e -> e.getSalary() > 50000)
    .map(e -> { e.setSalary(e.getSalary() * 1.1); return e; })
    .collect(Collectors.toList());
```

---

**Q218. Compare performance: sequential vs parallel stream.**

```
List<Integer> numbers = IntStream.rangeClosed(1, 10_000_000).boxed().collect(Collectors.toList());

long seqStart = System.currentTimeMillis();
long seqSum = numbers.stream().mapToLong(Integer::longValue).sum();
long seqTime = System.currentTimeMillis() - seqStart;

long parStart = System.currentTimeMillis();
long parSum = numbers.parallelStream().mapToLong(Integer::longValue).sum();
long parTime = System.currentTimeMillis() - parStart;

System.out.println("Sequential: " + seqTime + "ms, Parallel: " + parTime + "ms");
```

---

**Q219. Use parallelStream() for CPU-intensive operations.**

```
// Compute sum of squares for large list
long result = LongStream.rangeClosed(1, 1_000_000)
    .parallel()
    .map(n -> n * n)
    .sum();

System.out.println(result);
```

---

**Q220. When to use parallel streams and when not to.**

```
// Use parallel for large datasets and CPU-heavy work
List<Integer> largeList = IntStream.range(0, 1_000_000).boxed().collect(Collectors.toList());
long count = largeList.parallelStream().filter(n -> isPrime(n)).count();

// Avoid parallel for small collections (overhead > benefit)
List<Integer> smallList = Arrays.asList(1, 2, 3, 4, 5);
int sum = smallList.stream().mapToInt(n -> n).sum(); // use sequential here
```

---

**Q221. Handle thread safety in parallel streams.**

---

```
// WRONG – not thread-safe:
List<Integer> results = new ArrayList<>();
numbers.parallelStream().forEach(n -> results.add(n)); // race condition!

// CORRECT – use thread-safe collector:
List<Integer> safeResults = numbers.parallelStream()
    .filter(n -> n % 2 == 0)
    .collect(Collectors.toList()); // Collectors.toList() is thread-safe
```

---

**Q222. Understand parallel stream performance implications.**

```
// Parallel streams split data using Spliterator
// Best for: ArrayList, arrays (random access)
// Worse for: LinkedList (sequential access)

// Good candidate:
List<Integer> arrayList = new ArrayList<>(largeList);
long sumA = arrayList.parallelStream().mapToLong(n -> n).sum();

// Poor candidate:
LinkedList<Integer> linkedList = new LinkedList<>(largeList);
long sumL = linkedList.parallelStream().mapToLong(n -> n).sum(); // slower due to poor splitting
```

---

## 13. Employee Object Operations

### Basic Employee Operations

**Q223. Get list of all employee names.**

```
List<String> names = employees.stream()
    .map(Employee::getName)
    .collect(Collectors.toList());
```

**Q224. Get list of all employee IDs.**

```
List<Integer> ids = employees.stream()
    .map(Employee::getId)
    .collect(Collectors.toList());
```

**Q225. Filter employees by department.**

```
List<Employee> itEmployees = employees.stream()
    .filter(e -> e.getDept().equals("IT"))
    .collect(Collectors.toList());
```

**Q226. Filter employees by age greater than 30.**

```
List<Employee> result = employees.stream()
    .filter(e -> e.getAge() > 30)
    .collect(Collectors.toList());
```

**Q227. Filter employees by salary greater than 50000.**

```
List<Employee> result = employees.stream()
    .filter(e -> e.getSalary() > 50000)
    .collect(Collectors.toList());
```

---

**Q228. Filter employees by gender.**

```
List<Employee> females = employees.stream()
    .filter(e -> e.getGender().equals("Female"))
    .collect(Collectors.toList());
```

---

**Q229. Filter employees who joined after a specific year.**

```
List<Employee> result = employees.stream()
    .filter(e -> e.getJoiningDate().getYear() > 2018)
    .collect(Collectors.toList());
```

---

**Q230. Filter employees with age between 25 and 35.**

```
List<Employee> result = employees.stream()
    .filter(e -> e.getAge() >= 25 && e.getAge() <= 35)
    .collect(Collectors.toList());
```

---

**Q231. Get distinct department names.**

```
List<String> depts = employees.stream()
    .map(Employee::getDept)
    .distinct()
    .collect(Collectors.toList());
```

---

**Q232. Count total number of employees.**

```
long count = employees.stream().count();
System.out.println(count);
```

---

## Employee Salary Operations

**Q233. Find the highest paid employee.**

```
Optional<Employee> highest = employees.stream()
    .max(Comparator.comparingDouble(Employee::getSalary));
highest.ifPresent(e -> System.out.println(e.getName()));
```

---

**Q234. Find the lowest paid employee.**

```
Optional<Employee> lowest = employees.stream()
    .min(Comparator.comparingDouble(Employee::getSalary));
lowest.ifPresent(e -> System.out.println(e.getName()));
```

---

**Q235. Find the second highest salary.**

```
Optional<Double> secondHighest = employees.stream()
    .map(Employee::getSalary)
    .distinct()
    .sorted(Collections.reverseOrder())
    .skip(1)
    .findFirst();
System.out.println(secondHighest.orElse(0.0));
```

---

**Q236. Find the nth highest salary.**

```
int n = 3;
Optional<Double> nthHighest = employees.stream()
    .map(Employee::getSalary)
    .distinct()
    .sorted(Collections.reverseOrder())
    .skip(n - 1)
    .findFirst();
System.out.println(nthHighest.orElse(0.0));
```

---

**Q237. Find employees with salary above average.**

```
double avg = employees.stream()
    .mapToDouble(Employee::getSalary)
    .average()
    .orElse(0.0);

List<Employee> aboveAvg = employees.stream()
    .filter(e -> e.getSalary() > avg)
    .collect(Collectors.toList());
```

---

**Q238. Find employees with salary below average.**

```
double avg = employees.stream()
    .mapToDouble(Employee::getSalary)
    .average()
    .orElse(0.0);

List<Employee> belowAvg = employees.stream()
    .filter(e -> e.getSalary() < avg)
    .collect(Collectors.toList());
```

---

**Q239. Calculate total salary of all employees.**

```
double totalSalary = employees.stream()
    .mapToDouble(Employee::getSalary)
    .sum();
System.out.println(totalSalary);
```

---

**Q240. Calculate average salary of all employees.**

```
OptionalDouble avg = employees.stream()
    .mapToDouble(Employee::getSalary)
    .average();
System.out.println(avg.orElse(0.0));
```

---

**Q241. Find the salary range (min to max).**

```
DoubleSummaryStatistics stats = employees.stream()
    .mapToDouble(Employee::getSalary)
    .summaryStatistics();
System.out.println("Min: " + stats.getMin() + ", Max: " + stats.getMax());
```

---

## Department-wise Operations

**Q242. Group employees by department.**

```
Map<String, List<Employee>> byDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept));
```

**Q243. Count employees in each department.**

```
Map<String, Long> count = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept, Collectors.counting()));
```

**Q244. Find department with maximum employees.**

```
String maxDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept, Collectors.counting()))
    .entrySet().stream()
    .max((a, b) -> Long.compare(a.getValue(), b.getValue()))
    .map(Map.Entry::getKey)
    .orElse("None");
```

**Q245. Find department with minimum employees.**

```
String minDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept, Collectors.counting()))
    .entrySet().stream()
    .min((a, b) -> Long.compare(a.getValue(), b.getValue()))
    .map(Map.Entry::getKey)
    .orElse("None");
```

**Q246. Find average salary per department.**

```
Map<String, Double> avgByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept,
        Collectors.averagingDouble(Employee::getSalary)));
```

**Q247. Find total salary per department.**

```
Map<String, Double> totalByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept,
        Collectors.summingDouble(Employee::getSalary)));
```

**Q248. Find highest paid employee in each department.**

---



```
Map<String, Optional<Employee>> highestByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept,
        Collectors.maxBy(Comparator.comparingDouble(Employee::getSalary))));
```

---

**Q249. Find lowest paid employee in each department.**

```
Map<String, Optional<Employee>> lowestByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept,
        Collectors.minBy(Comparator.comparingDouble(Employee::getSalary))));
```

---

**Q250. Find second highest paid employee in each department.**

```
Map<String, Optional<Employee>> result = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDept,
        Collectors.collectingAndThen(
            Collectors.toList(),
            list -> list.stream()
                .sorted(Comparator.comparingDouble(Employee::getSalary).reversed())
                .skip(1)
                .findFirst()
        )
    ));
```

---

**Q251. Sort employees by salary within each department.**

```
Map<String, List<Employee>> sorted = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDept,
        Collectors.collectingAndThen(
            Collectors.toList(),
            list -> list.stream()
                .sorted(Comparator.comparingDouble(Employee::getSalary))
                .collect(Collectors.toList())
        )
    ));
```

---

**Q252. List employee names grouped by department.**

```
Map<String, List<String>> namesByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDept,
        Collectors.mapping(Employee::getName, Collectors.toList())
    ));
```

---

## Gender-based Operations

**Q253. Count male and female employees.**

```
Map<String, Long> countByGender = employees.stream()
    .collect(Collectors.groupingBy(Employee::getGender, Collectors.counting()));
```

---

**Q254. Find average age of male and female employees.**

```
Map<String, Double> avgAgeByGender = employees.stream()
    .collect(Collectors.groupingBy(Employee::getGender,
        Collectors.averagingInt(Employee::getAge)));
```

---

**Q255. Find average salary of male and female employees.**

```
Map<String, Double> avgSalaryByGender = employees.stream()
    .collect(Collectors.groupingBy(Employee::getGender,
        Collectors.averagingDouble(Employee::getSalary)));
```

---

**Q256. Find highest paid male employee.**

```
Optional<Employee> highestMale = employees.stream()
    .filter(e -> e.getGender().equals("Male"))
    .max(Comparator.comparingDouble(Employee::getSalary));
```

---

**Q257. Find highest paid female employee.**

```
Optional<Employee> highestFemale = employees.stream()
    .filter(e -> e.getGender().equals("Female"))
    .max(Comparator.comparingDouble(Employee::getSalary));
```

---

**Q258. Group employees by gender.**

```
Map<String, List<Employee>> byGender = employees.stream()
    .collect(Collectors.groupingBy(Employee::getGender));
```

---

**Q259. Find the youngest male employee.**

```
Optional<Employee> youngestMale = employees.stream()
    .filter(e -> e.getGender().equals("Male"))
    .min(Comparator.comparingInt(Employee::getAge));
```

---

**Q260. Find the youngest female employee.**

```
Optional<Employee> youngestFemale = employees.stream()
    .filter(e -> e.getGender().equals("Female"))
    .min(Comparator.comparingInt(Employee::getAge));
```

---

**Q261. Find the oldest employee in the organization.**

```
Optional<Employee> oldest = employees.stream()
    .max(Comparator.comparingInt(Employee::getAge));
oldest.ifPresent(e -> System.out.println(e.getName() + " - " + e.getAge()));
```

---

**Q262. Find the youngest employee in the organization.**

```
Optional<Employee> youngest = employees.stream()
    .min(Comparator.comparingInt(Employee::getAge));
youngest.ifPresent(e -> System.out.println(e.getName() + " - " + e.getAge()));
```

---

## Advanced Employee Queries

**Q263. Find employees whose name starts with a specific letter.**

```
List<Employee> result = employees.stream()
    .filter(e -> e.getName().startsWith("A"))
    .collect(Collectors.toList());
```

---

**Q264. Find employees with age less than 30 in HR department.**

```
List<Employee> result = employees.stream()
    .filter(e -> e.getAge() < 30 && e.getDept().equals("HR"))
    .collect(Collectors.toList());
```

---

**Q265. Find the most senior employee (earliest joining date).**

```
Optional<Employee> mostSenior = employees.stream()
    .min(Comparator.comparing(Employee::getJoiningDate));
mostSenior.ifPresent(e -> System.out.println(e.getName()));
```

---

**Q266. Find employees who joined in a specific year.**

```
int year = 2020;
List<Employee> result = employees.stream()
    .filter(e -> e.getJoiningDate().getYear() == year)
    .collect(Collectors.toList());
```

---

**Q267. Get top N highest paid employees.**

```
int n = 3;
List<Employee> topN = employees.stream()
    .sorted(Comparator.comparingDouble(Employee::getSalary).reversed())
    .limit(n)
    .collect(Collectors.toList());
```

---

**Q268. Get employee with maximum age in each department.**

```
Map<String, Optional<Employee>> result = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept,
        Collectors.maxBy(Comparator.comparingInt(Employee::getAge))));
```

---

**Q269. Find departments where average salary > 60000.**

```
List<String> depts = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept,
        Collectors.averagingDouble(Employee::getSalary)))
    .entrySet().stream()
    .filter(e -> e.getValue() > 60000)
    .map(Map.Entry::getKey)
    .collect(Collectors.toList());
```

---

**Q270. Find employees in multiple departments.**

```
List<String> targetDepts = Arrays.asList("IT", "Finance");
List<Employee> result = employees.stream()
    .filter(e -> targetDepts.contains(e.getDept()))
    .collect(Collectors.toList());
```

---

**Q271. Find employees whose salary is in top 10%.**

```
DoubleSummaryStatistics stats = employees.stream()
    .mapToDouble(Employee::getSalary).summaryStatistics();
double threshold = stats.getMax() * 0.90;

List<Employee> top10 = employees.stream()
    .filter(e -> e.getSalary() >= threshold)
    .collect(Collectors.toList());
```

---

**Q272. Create a salary increment report (increase by 10%).**

```
List<String> report = employees.stream()
    .map(e -> e.getName() + ": " + e.getSalary() + " -> " + (e.getSalary() * 1.10))
    .collect(Collectors.toList());
report.forEach(System.out::println);
```

---

**Q273. Find employee with maximum experience.**

```
Optional<Employee> mostExperienced = employees.stream()
    .min(Comparator.comparing(Employee::getJoiningDate)); // earliest join = most experience
```

---

**Q274. List all unique cities where employees are located.**

```
List<String> cities = employees.stream()
    .map(Employee::getCity)
    .distinct()
    .sorted()
    .collect(Collectors.toList());
```

---

**Q275. Find employees located in a specific city and sort by name.**

```
List<Employee> result = employees.stream()
    .filter(e -> e.getCity().equals("Mumbai"))
    .sorted(Comparator.comparing(Employee::getName))
    .collect(Collectors.toList());
```

---

## 14. Map & Entry Operations

---

### Map Creation & Manipulation

**Q276. Convert list to map with custom key-value.**

```
Map<String, Double> nameToSalary = employees.stream()
    .collect(Collectors.toMap(Employee::getName, Employee::getSalary));
```

---

**Q277. Create a frequency map from a list.**

```
List<String> words = Arrays.asList("apple", "banana", "apple", "cherry", "banana", "apple");
Map<String, Long> freq = words.stream()
    .collect(Collectors.groupingBy(s -> s, Collectors.counting()));
// {apple=3, banana=2, cherry=1}
```

---

**Q278. Merge two maps using streams.**

```
Map<String, Integer> map1 = Map.of("a", 1, "b", 2);
Map<String, Integer> map2 = Map.of("b", 3, "c", 4);

Map<String, Integer> merged = Stream.concat(map1.entrySet().stream(), map2.entrySet().stream())
    .collect(Collectors.toMap(
        Map.Entry::getKey,
        Map.Entry::getValue,
        (v1, v2) -> v1 + v2 // merge strategy: sum values
    ));
// {a=1, b=5, c=4}
```

---

**Q279. Filter map entries based on key.**

```
Map<String, Integer> map = Map.of("Alice", 1, "Bob", 2, "Anna", 3);
Map<String, Integer> result = map.entrySet().stream()
    .filter(e -> e.getKey().startsWith("A"))
    .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue));
// {Alice=1, Anna=3}
```

---

**Q280. Filter map entries based on value.**

```
Map<String, Integer> salaries = Map.of("Alice", 70000, "Bob", 40000, "Charlie", 90000);
Map<String, Integer> highEarnings = salaries.entrySet().stream()
    .filter(e -> e.getValue() > 50000)
    .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue));
// {Alice=70000, Charlie=90000}
```

---

**Q281. Convert map keys to uppercase.**

```
Map<String, Integer> map = Map.of("alice", 1, "bob", 2);
Map<String, Integer> result = map.entrySet().stream()
    .collect(Collectors.toMap(
        e -> e.getKey().toUpperCase(),
        Map.Entry::getValue
    ));
// {ALICE=1, BOB=2}
```

---

**Q282. Convert map values to uppercase.**

```
Map<String, String> map = Map.of("name", "alice", "city", "mumbai");
Map<String, String> result = map.entrySet().stream()
    .collect(Collectors.toMap(
        Map.Entry::getKey,
        e -> e.getValue().toUpperCase()
    ));
```

---

**Q283. Swap keys and values in a map.**

```
Map<String, Integer> map = Map.of("Alice", 1, "Bob", 2);
Map<Integer, String> swapped = map.entrySet().stream()
    .collect(Collectors.toMap(Map.Entry::getValue, Map.Entry::getKey));
// {1=Alice, 2=Bob}
```

---

**Q284. Sort map by keys.**

```
Map<String, Integer> map = new HashMap<>(Map.of("Charlie", 3, "Alice", 1, "Bob", 2));
Map<String, Integer> sorted = map.entrySet().stream()
    .sorted(Map.Entry.comparingByKey())
    .collect(Collectors.toMap(
        Map.Entry::getKey,
        Map.Entry::getValue,
        (e1, e2) -> e1,
        LinkedHashMap::new
    ));
```

---

**Q285. Sort map by values.**

```
Map<String, Integer> map = Map.of("Charlie", 3, "Alice", 1, "Bob", 2);
Map<String, Integer> sorted = map.entrySet().stream()
    .sorted(Map.Entry.comparingByValue())
    .collect(Collectors.toMap(
        Map.Entry::getKey,
        Map.Entry::getValue,
        (e1, e2) -> e1,
        LinkedHashMap::new
    ));
```

---

**Q286. Find the entry with maximum value in a map.**

```
Map<String, Integer> map = Map.of("Alice", 90, "Bob", 75, "Charlie", 85);
Map.Entry<String, Integer> maxEntry = map.entrySet().stream()
    .max(Map.Entry.comparingByValue())
    .orElseThrow();
System.out.println(maxEntry.getKey() + ": " + maxEntry.getValue()); // Alice: 90
```

---

**Q287. Find the entry with minimum value in a map.**

```
Map<String, Integer> map = Map.of("Alice", 90, "Bob", 75, "Charlie", 85);
Map.Entry<String, Integer> minEntry = map.entrySet().stream()
    .min(Map.Entry.comparingByValue())
    .orElseThrow();
System.out.println(minEntry.getKey() + ": " + minEntry.getValue()); // Bob: 75
```

---

#### Q288. Sum all values in a map.

```
Map<String, Integer> map = Map.of("a", 10, "b", 20, "c", 30);
int sum = map.values().stream()
    .mapToInt(Integer::intValue)
    .sum();
System.out.println(sum); // 60
```

---

#### Q289. Get top N entries from a map based on values.

```
Map<String, Integer> scores = Map.of("Alice", 90, "Bob", 75, "Charlie", 85, "David", 95);
int n = 2;
List<Map.Entry<String, Integer>> topN = scores.entrySet().stream()
    .sorted(Map.Entry.<String, Integer>comparingByValue().reversed())
    .limit(n)
    .collect(Collectors.toList());
// [David=95, Alice=90]
```

---

## Map to List Conversions

#### Q290. Convert map to list of "key=value" strings.

```
Map<String, Integer> map = Map.of("Alice", 90, "Bob", 75);
List<String> result = map.entrySet().stream()
    .map(e -> e.getKey() + "=" + e.getValue())
    .collect(Collectors.toList());
// [Alice=90, Bob=75]
```

---

#### Q291. Convert map entries to list of custom objects.

```
Map<String, Double> salaryMap = Map.of("Alice", 70000.0, "Bob", 60000.0);
List<Employee> emplList = salaryMap.entrySet().stream()
    .map(e -> new Employee(e.getKey(), e.getValue()))
    .collect(Collectors.toList());
```

---

#### Q292. Extract all keys from map as a list.

```
Map<String, Integer> map = Map.of("Alice", 1, "Bob", 2, "Charlie", 3);
List<String> keys = new ArrayList<>(map.keySet());
// Or:
List<String> keyList = map.entrySet().stream()
    .map(Map.Entry::getKey)
    .collect(Collectors.toList());
```

---

#### Q293. Extract all values from map as a list.

```
Map<String, Integer> map = Map.of("Alice", 1, "Bob", 2, "Charlie", 3);
List<Integer> values = map.values().stream()
    .collect(Collectors.toList());
```

---

#### Q294. Filter map and collect to list.

```
Map<String, Integer> scores = Map.of("Alice", 90, "Bob", 55, "Charlie", 70);
List<String> passed = scores.entrySet().stream()
    .filter(e -> e.getValue() >= 60)
    .map(Map.Entry::getKey)
    .collect(Collectors.toList());
// [Alice, Charlie]
```

---

## 15. Array & List Conversions

### Array Operations

#### Q295. Convert array to list using streams.

```
String[] arr = {"apple", "banana", "cherry"};
List<String> list = Arrays.stream(arr).collect(Collectors.toList());
```

---

#### Q296. Convert array to set using streams.

```
String[] arr = {"apple", "banana", "apple", "cherry"};
Set<String> set = Arrays.stream(arr).collect(Collectors.toSet());
// {apple, banana, cherry}
```

---

#### Q297. Filter array elements using streams.

```
int[] arr = {1, 2, 3, 4, 5, 6};
int[] evens = Arrays.stream(arr)
    .filter(n -> n % 2 == 0)
    .toArray();
// [2, 4, 6]
```

---

#### Q298. Sort an array using streams.

```
int[] arr = {5, 2, 8, 1, 9};
int[] sorted = Arrays.stream(arr).sorted().toArray();
// [1, 2, 5, 8, 9]
```

---

#### Q299. Find max element in an array using streams.

```
int[] arr = {5, 2, 8, 1, 9};
int max = Arrays.stream(arr).max().getAsInt();
System.out.println(max); // 9
```

---

#### Q300. Find min element in an array using streams.



```
int[] arr = {5, 2, 8, 1, 9};
int min = Arrays.stream(arr).min().getAsInt();
System.out.println(min); // 1
```

---

## 15. Array & List Conversions

### Array Operations

**Q301. Convert primitive array to list.**

```
int[] arr = {1, 2, 3, 4, 5};
// Primitive arrays can't be directly collected – use boxed()
List<Integer> list = Arrays.stream(arr)
    .boxed()
    .collect(Collectors.toList());

// [1, 2, 3, 4, 5]
```

**Q302. Convert list to primitive array.**

```
List<Integer> list = Arrays.asList(1, 2, 3, 4, 5);
int[] arr = list.stream()
    .mapToInt(Integer::intValue)
    .toArray();
```

**Q303. Find sum of array elements.**

```
int[] arr = {1, 2, 3, 4, 5};
int sum = Arrays.stream(arr).sum();
System.out.println(sum); // 15
```

**Q304. Find average of array elements.**

```
int[] arr = {1, 2, 3, 4, 5};
double avg = Arrays.stream(arr).average().getAsDouble();
System.out.println(avg); // 3.0
```

### IntStream, LongStream, DoubleStream

**Q305. Create IntStream from range.**

```
// rangeClosed includes both endpoints
IntStream.rangeClosed(1, 10).forEach(System.out::println); // 1..10

// range excludes end
IntStream.range(1, 10).forEach(System.out::println); // 1..9
```

**Q306. Use IntStream to generate numbers.**

```
// Generate first 10 even numbers
IntStream.iterate(0, n -> n + 2)
    .limit(10)
    .forEach(System.out::println);
// 0, 2, 4, 6, 8, 10, 12, 14, 16, 18
```

---

#### Q307. Convert IntStream to List.

```
List<Integer> list = IntStream.rangeClosed(1, 5)
    .boxed()
    .collect(Collectors.toList());

// [1, 2, 3, 4, 5]
```

---

#### Q308. Use mapToInt for primitive operations.

```
List<String> words = Arrays.asList("apple", "banana", "cherry");
int totalChars = words.stream()
    .mapToInt(String::length)
    .sum();
System.out.println(totalChars); // 5+6+6 = 17
```

---

#### Q309. Use mapToDouble for decimal operations.

```
List<Employee> employees = getEmployees();
double avgSalary = employees.stream()
    .mapToDouble(Employee::getSalary)
    .average()
    .orElse(0.0);
```

---

#### Q310. Calculate sum using IntStream.

```
int sum = IntStream.rangeClosed(1, 100).sum();
System.out.println(sum); // 5050
```

---

#### Q311. Find max using IntStream.

```
int max = IntStream.of(3, 7, 1, 9, 4).max().getAsInt();
System.out.println(max); // 9
```

---

#### Q312. Use summaryStatistics with IntStream.

```
IntSummaryStatistics stats = IntStream.of(1, 2, 3, 4, 5).summaryStatistics();
System.out.println("Count: " + stats.getCount());
System.out.println("Sum: " + stats.getSum());
System.out.println("Min: " + stats.getMin());
System.out.println("Max: " + stats.getMax());
System.out.println("Avg: " + stats.getAverage());
```

---

## 16. Optional Operations

---

### Q313. Use `Optional.of()` vs `Optional.ofNullable()`.

```
// Optional.of() – throws NullPointerException if value is null
Optional<String> opt1 = Optional.of("Hello"); // OK
// Optional.of(null); // throws NPE!

// Optional.ofNullable() – safe with null
Optional<String> opt2 = Optional.ofNullable(null); // Optional.empty
Optional<String> opt3 = Optional.ofNullable("World"); // Optional[World]
```

---

### Q314. Get value from `Optional` or provide default.

```
Optional<String> opt = Optional.ofNullable(null);
String value = opt.orElse("Default Value");
System.out.println(value); // Default Value
```

---

### Q315. Use `orElse()` vs `orElseGet()`.

```
Optional<String> opt = Optional.empty();

// orElse – evaluates the default value EAGERLY
String val1 = opt.orElse("default");

// orElseGet – evaluates the supplier LAZILY (only if empty)
String val2 = opt.orElseGet(() -> "computed default");

// Prefer orElseGet when default computation is expensive
```

---

### Q316. Use `orElseThrow()` to throw exception.

```
Optional<Employee> emp = employees.stream()
    .filter(e -> e.getId() == 999)
    .findFirst();

Employee found = emp.orElseThrow(() -> new RuntimeException("Employee not found"));
```

---

### Q317. Check if `Optional` is present.

```
Optional<String> opt = Optional.ofNullable("Hello");

if (opt.isPresent()) {
    System.out.println(opt.get()); // Hello
}

// Cleaner alternative:
opt.ifPresent(System.out::println);
```

---

### Q318. Filter `Optional` values.

```
Optional<String> opt = Optional.of("Hello World");
Optional<String> filtered = opt.filter(s -> s.contains("World"));
System.out.println(filtered.isPresent()); // true
```

---

### Q319. Map Optional values.

```
Optional<String> opt = Optional.of("hello");
Optional<String> upperOpt = opt.map(String::toUpperCase);
System.out.println(upperOpt.get()); // HELLO
```

---

### Q320. FlatMap with Optional.

```
Optional<String> opt = Optional.of("hello");
// Use flatMap when the mapping function returns Optional
Optional<String> result = opt.flatMap(s -> s.isEmpty() ? Optional.empty() : Optional.of(s.toUpperCase()));
System.out.println(result.get()); // HELLO
```

---

### Q321. Chain multiple Optional operations.

```
Optional<Employee> emp = findEmployeeById(101);
String city = emp.map(Employee::getAddress)
                 .map(Address::getCity)
                 .orElse("Unknown");
System.out.println(city);
```

---

### Q322. Convert null to Optional and handle gracefully.

```
String name = null; // could be null
String result = Optional.ofNullable(name)
                        .map(String::toUpperCase)
                        .orElse("NO NAME");
System.out.println(result); // NO NAME
```

---

## 17. Partitioning

---

### Q323. Partition list into two groups based on condition.

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8);
Map<Boolean, List<Integer>> partitioned = numbers.stream()
        .collect(Collectors.partitioningBy(n -> n % 2 == 0));
// {true=[2, 4, 6, 8], false=[1, 3, 5, 7]}
```

---

### Q324. Partition employees into juniors and seniors based on age.

```
Map<Boolean, List<Employee>> partitioned = employees.stream()
        .collect(Collectors.partitioningBy(e -> e.getAge() >= 30));
List<Employee> seniors = partitioned.get(true);
List<Employee> juniors = partitioned.get(false);
```

---

### Q325. Partition products into expensive and cheap.

```
Map<Boolean, List<Product>> partitioned = products.stream()
        .collect(Collectors.partitioningBy(p -> p.getPrice() > 1000));
```

---

### Q326. Partition numbers into positive and negative.

```
List<Integer> numbers = Arrays.asList(-3, -1, 0, 2, 5, -7, 8);
Map<Boolean, List<Integer>> partitioned = numbers.stream()
    .collect(Collectors.partitioningBy(n -> n >= 0));
// {true=[0, 2, 5, 8], false=[-3, -1, -7]}
```

---

**Q327. Get counts of both partitions.**

```
Map<Boolean, Long> counts = employees.stream()
    .collect(Collectors.partitioningBy(
        e -> e.getSalary() > 50000,
        Collectors.counting()
    ));
System.out.println("High earners: " + counts.get(true));
System.out.println("Others: " + counts.get(false));
```

---

**Q328. Process both partitions separately.**

```
Map<Boolean, List<Employee>> partitioned = employees.stream()
    .collect(Collectors.partitioningBy(e -> e.getAge() >= 30));

// Process seniors
partitioned.get(true).forEach(e -> System.out.println("Senior: " + e.getName()));

// Process juniors
partitioned.get(false).forEach(e -> System.out.println("Junior: " + e.getName()));
```

---

## 18. Statistical Operations

---

**Q329. Get summary statistics for a list of integers.**

```
List<Integer> numbers = Arrays.asList(3, 7, 1, 9, 4, 6);
IntSummaryStatistics stats = numbers.stream()
    .mapToInt(Integer::intValue)
    .summaryStatistics();
System.out.println(stats);
// IntSummaryStatistics{count=6, sum=30, min=1, average=5.0, max=9}
```

---

**Q330. Calculate count, sum, min, max, and average in one operation.**

```
List<Integer> numbers = Arrays.asList(10, 20, 30, 40, 50);
IntSummaryStatistics stats = numbers.stream()
    .mapToInt(n -> n)
    .summaryStatistics();
System.out.println("Count: " + stats.getCount());
System.out.println("Sum: " + stats.getSum());
System.out.println("Min: " + stats.getMin());
System.out.println("Max: " + stats.getMax());
System.out.println("Average: " + stats.getAverage());
```

---

**Q331. Get statistics for employee salaries.**

```
DoubleSummaryStatistics salaryStats = employees.stream()
    .mapToDouble(Employee::getSalary)
    .summaryStatistics();
System.out.println("Min Salary: " + salaryStats.getMin());
System.out.println("Max Salary: " + salaryStats.getMax());
System.out.println("Avg Salary: " + salaryStats.getAverage());
System.out.println("Total: " + salaryStats.getSum());
```

---

#### Q332. Use DoubleSummaryStatistics.

```
List<Double> prices = Arrays.asList(10.5, 20.3, 15.8, 8.9, 30.0);
DoubleSummaryStatistics stats = prices.stream()
    .mapToDouble(Double::doubleValue)
    .summaryStatistics();

System.out.println(stats);
```

---

#### Q333. Use IntSummaryStatistics.

```
List<Employee> employees = getEmployees();
IntSummaryStatistics ageStats = employees.stream()
    .mapToInt(Employee::getAge)
    .summaryStatistics();

System.out.println("Youngest: " + ageStats.getMin());
System.out.println("Oldest: " + ageStats.getMax());
System.out.println("Average Age: " + ageStats.getAverage());
```

---

#### Q334. Use LongSummaryStatistics.

```
List<Long> ids = Arrays.asList(100L, 200L, 300L, 400L, 500L);
LongSummaryStatistics stats = ids.stream()
    .mapToLong(Long::longValue)
    .summaryStatistics();

System.out.println(stats);
```

---

#### Q335. Get statistics per group (e.g., per department).

```
Map<String, DoubleSummaryStatistics> statsByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDept,
        Collectors.summarizingDouble(Employee::getSalary)
    ));

statsByDept.forEach((dept, stats) ->
    System.out.println(dept + ": avg=" + stats.getAverage() + ", max=" + stats.getMax()));
```

---

## 19. File Operations with Streams

### Reading Files

#### Q336. Read all lines from a file using streams.

```
Path path = Paths.get("data.txt");
try (Stream<String> lines = Files.lines(path)) {
    lines.forEach(System.out::println);
}
```

---

**Q337. Count number of lines in a file.**

```
Path path = Paths.get("data.txt");
try (Stream<String> lines = Files.lines(path)) {
    long count = lines.count();
    System.out.println("Lines: " + count);
}
```

---

**Q338. Find lines containing a specific word.**

```
Path path = Paths.get("data.txt");
try (Stream<String> lines = Files.lines(path)) {
    List<String> matches = lines
        .filter(line -> line.contains("error"))
        .collect(Collectors.toList());
    matches.forEach(System.out::println);
}
```

---

**Q339. Count unique words in a file.**

```
Path path = Paths.get("data.txt");
try (Stream<String> lines = Files.lines(path)) {
    long uniqueWords = lines
        .flatMap(line -> Arrays.stream(line.split("\\s+")))
        .map(String::toLowerCase)
        .distinct()
        .count();
    System.out.println("Unique words: " + uniqueWords);
}
```

---

**Q340. Find the longest line in a file.**

```
Path path = Paths.get("data.txt");
try (Stream<String> lines = Files.lines(path)) {
    Optional<String> longest = lines
        .max(Comparator.comparingInt(String::length));
    longest.ifPresent(System.out::println);
}
```

---

**Q341. Convert file content to uppercase.**

```
Path input = Paths.get("data.txt");
try (Stream<String> lines = Files.lines(input)) {
    List<String> upperLines = lines
        .map(String::toUpperCase)
        .collect(Collectors.toList());
    Files.write(Paths.get("upper.txt"), upperLines);
}
```

---

#### Q342. Filter empty lines from a file.

```
Path path = Paths.get("data.txt");
try (Stream<String> lines = Files.lines(path)) {
    List<String> nonEmpty = lines
        .filter(line -> !line.isBlank())
        .collect(Collectors.toList());
}
```

---

#### Q343. Count occurrences of a word in a file.

```
String targetWord = "java";
Path path = Paths.get("data.txt");
try (Stream<String> lines = Files.lines(path)) {
    long count = lines
        .flatMap(line -> Arrays.stream(line.split("\\s+")))
        .map(String::toLowerCase)
        .filter(w -> w.equals(targetWord))
        .count();
    System.out.println("Occurrences of '" + targetWord + "': " + count);
}
```

---

### CSV Processing

#### Q344. Read and parse CSV file using streams.

```
Path path = Paths.get("employees.csv");
try (Stream<String> lines = Files.lines(path)) {
    List<String[]> rows = lines
        .skip(1) // skip header
        .map(line -> line.split(","))
        .collect(Collectors.toList());
    rows.forEach(row -> System.out.println(Arrays.toString(row)));
}
```

---

#### Q345. Filter rows in CSV based on condition.

```
Path path = Paths.get("employees.csv");
try (Stream<String> lines = Files.lines(path)) {
    List<String> highSalary = lines
        .skip(1)
        .filter(line -> {
            String[] parts = line.split(",");
            return Double.parseDouble(parts[2]) > 50000;
        })
        .collect(Collectors.toList());
}
```

---

#### Q346. Group CSV data by a column.

```
Path path = Paths.get("employees.csv");
try (Stream<String> lines = Files.lines(path)) {
    Map<String, List<String>> grouped = lines
        .skip(1)
        .collect(Collectors.groupingBy(line -> line.split(",")[1])); // group by dept (col 1)
}
```



---

### Q347. Calculate aggregate statistics from CSV.

```
Path path = Paths.get("employees.csv");
try (Stream<String> lines = Files.lines(path)) {
    DoubleSummaryStatistics stats = lines
        .skip(1)
        .mapToDouble(line -> Double.parseDouble(line.split(",")[2]))
        .summaryStatistics();
    System.out.println("Avg salary: " + stats.getAverage());
}
```

---

### Q348. Convert CSV to list of objects.

```
Path path = Paths.get("employees.csv");
try (Stream<String> lines = Files.lines(path)) {
    List<Employee> employees = lines
        .skip(1)
        .map(line -> {
            String[] parts = line.split(",");
            return new Employee(parts[0], parts[1], Double.parseDouble(parts[2]));
        })
        .collect(Collectors.toList());
}
```

---

### Q349. Extract specific columns from CSV.

```
Path path = Paths.get("employees.csv");
try (Stream<String> lines = Files.lines(path)) {
    List<String> names = lines
        .skip(1)
        .map(line -> line.split(",")[0]) // extract name column
        .collect(Collectors.toList());
}
```

---

## 20. Advanced & Complex Scenarios

### Complex Combinations

#### Q350. Find pairs of numbers from a list that sum to a specific value.

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9);
int target = 10;
List<String> pairs = numbers.stream()
    .flatMap(n1 -> numbers.stream()
        .filter(n2 -> n1 < n2 && n1 + n2 == target)
        .map(n2 -> "(" + n1 + ", " + n2 + ")")
    )
    .collect(Collectors.toList());
// [(1, 9), (2, 8), (3, 7), (4, 6)]
```

---

#### Q351. Find all permutations using streams.

```
// Simple: all pairs from a list
List<Integer> list = Arrays.asList(1, 2, 3);
List<String> perms = list.stream()
    .flatMap(a -> list.stream()
        .filter(b -> !b.equals(a))
        .map(b -> a + " " + b))
    .collect(Collectors.toList());
// [12, 13, 21, 23, 31, 32]
```

---

#### Q352. Find combinations of elements.

```
List<Integer> list = Arrays.asList(1, 2, 3, 4);
// All pairs (combinations of 2)
List<String> combos = list.stream()
    .flatMap(a -> list.stream()
        .filter(b -> a < b)
        .map(b -> "(" + a + ", " + b + ")"))
    .collect(Collectors.toList());
// [(1,2), (1,3), (1,4), (2,3), (2,4), (3,4)]
```

---

#### Q353. Implement custom collector.

```
// Custom collector that joins strings with '|' and wraps in brackets
Collector<String, StringBuilder, String> customCollector = Collector.of(
    StringBuilder::new, // supplier
    (sb, s) -> { if (sb.length() > 0) sb.append("|"); sb.append(s); }, // accumulator
    StringBuilder::append, // combiner
    sb -> "[" + sb.toString() + "]" // finisher
);

String result = Stream.of("A", "B", "C").collect(customCollector);
System.out.println(result); // [A|B|C]
```

---

#### Q354. Chain multiple stream operations.

```
List<String> result = employees.stream()
    .filter(e -> e.getDept().equals("IT"))
    .filter(e -> e.getSalary() > 50000)
    .map(Employee::getName)
    .map(String::toUpperCase)
    .sorted()
    .distinct()
    .collect(Collectors.toList());
```

---

#### Q355. Use peek() for debugging streams.

```
List<Integer> result = Arrays.asList(1, 2, 3, 4, 5).stream()
    .peek(n -> System.out.println("Before filter: " + n))
    .filter(n -> n % 2 == 0)
    .peek(n -> System.out.println("After filter: " + n))
    .map(n -> n * 2)
    .peek(n -> System.out.println("After map: " + n))
    .collect(Collectors.toList());
```

---

#### Q356. Limit and skip operations.

```
List<Integer> numbers = IntStream.rangeClosed(1, 20).boxed().collect(Collectors.toList());

// First 5 elements
List<Integer> first5 = numbers.stream().limit(5).collect(Collectors.toList());
// [1, 2, 3, 4, 5]

// Elements 6-10 (skip 5, take 5)
List<Integer> page2 = numbers.stream().skip(5).limit(5).collect(Collectors.toList());
// [6, 7, 8, 9, 10]
```

---

#### Q357. Create infinite streams and control them.

```
// Infinite stream of natural numbers
Stream.iterate(1, n -> n + 1)
    .limit(10)
    .forEach(System.out::println);

// Infinite stream of random numbers
Stream.generate(Math::random)
    .limit(5)
    .forEach(System.out::println);
```

---

### Real-world Scenarios

#### Q358. Process JSON data using streams.

```
// Assume a list of JSON-like maps
List<Map<String, Object>> records = getJsonData();
List<String> names = records.stream()
    .filter(r -> (Integer) r.get("age") > 25)
    .map(r -> (String) r.get("name"))
    .collect(Collectors.toList());
```

---

#### Q359. Handle exceptions in stream operations.

```
// Wrap checked exception in unchecked
List<Integer> result = strNumbers.stream()
    .map(s -> {
        try {
            return Integer.parseInt(s);
        } catch (NumberFormatException e) {
            return 0; // default value on error
        }
    })
    .collect(Collectors.toList());
```

---

#### Q360. Convert between different date formats using streams.

```
List<String> dates = Arrays.asList("2024-01-15", "2024-06-20", "2024-12-01");
DateTimeFormatter in = DateTimeFormatter.ofPattern("yyyy-MM-dd");
DateTimeFormatter out = DateTimeFormatter.ofPattern("dd/MM/yyyy");

List<String> formatted = dates.stream()
    .map(d -> LocalDate.parse(d, in).format(out))
    .collect(Collectors.toList());
// [15/01/2024, 20/06/2024, 01/12/2024]
```

---

**Q361. Validate data using streams.**

```
List<String> emails = Arrays.asList("user@mail.com", "invalid", "test@test.org");
String emailRegex = "^\\\\w.-]+@[\\\\w.-]+\\\\.[a-z]{2,}$";

Map<Boolean, List<String>> validated = emails.stream()
    .collect(Collectors.partitioningBy(e -> e.matches(emailRegex)));

System.out.println("Valid: " + validated.get(true));
System.out.println("Invalid: " + validated.get(false));
```

---

**Q362. Implement caching with streams.**

```
// Build a cached lookup map from a list
Map<Integer, Employee> cache = employees.stream()
    .collect(Collectors.toMap(Employee::getId, e -> e));

// Later, look up quickly
Employee emp = cache.get(101);
```

---

**Q363. Batch process elements (e.g., process 100 at a time).**

```
List<Integer> numbers = IntStream.rangeClosed(1, 500).boxed().collect(Collectors.toList());
int batchSize = 100;

IntStream.range(0, (numbers.size() + batchSize - 1) / batchSize)
    .forEach(i -> {
        List<Integer> batch = numbers.subList(
            i * batchSize,
            Math.min((i + 1) * batchSize, numbers.size())
        );
        System.out.println("Processing batch " + (i + 1) + ": " + batch.size() + " items");
    });
```

---

**Q364. Compare two lists and find differences.**

```
List<String> list1 = Arrays.asList("Alice", "Bob", "Charlie", "David");
List<String> list2 = Arrays.asList("Bob", "David", "Eve");

// In list1 but not list2
List<String> onlyInList1 = list1.stream()
    .filter(s -> !list2.contains(s))
    .collect(Collectors.toList()); // [Alice, Charlie]

// In list2 but not list1
List<String> onlyInList2 = list2.stream()
    .filter(s -> !list1.contains(s))
    .collect(Collectors.toList()); // [Eve]
```

---

**Q365. Implement pagination using streams.**

```
List<Employee> employees = getEmployees(); // large list
int page = 2;           // page number (1-based)
int pageSize = 10;

List<Employee> pageResult = employees.stream()
    .skip((long)(page - 1) * pageSize)
    .limit(pageSize)
    .collect(Collectors.toList());
```

---

## Performance & Optimization

### Q366. When to use sequential vs parallel streams.

```
// Use sequential for:
// - Small collections (< 10,000 elements)
// - Operations with side effects
// - Ordered output is critical
List<Integer> small = Arrays.asList(1, 2, 3, 4, 5);
int sum = small.stream().mapToInt(n -> n).sum();

// Use parallel for:
// - Large collections with CPU-heavy independent operations
List<Integer> large = IntStream.rangeClosed(1, 1_000_000).boxed().collect(Collectors.toList());
long count = large.parallelStream().filter(n -> isPrime(n)).count();
```

---

### Q367. Avoid using streams for small collections.

```
// Overhead not worth it for tiny lists – plain loop is faster
List<Integer> tiny = Arrays.asList(1, 2, 3);
int sum = 0;
for (int n : tiny) sum += n; // better than stream for tiny lists

// Stream is fine for readability even on small lists in non-hot paths
int streamSum = tiny.stream().mapToInt(n -> n).sum();
```

---

### Q368. Understand laziness in streams.

```
// Intermediate operations are lazy – they don't execute until a terminal op
Stream<Integer> lazyStream = Stream.of(1, 2, 3, 4, 5)
    .filter(n -> { System.out.println("filter: " + n); return n % 2 == 0; })
    .map(n -> { System.out.println("map: " + n); return n * 2; });

// Nothing has printed yet!
System.out.println("Before terminal");
lazyStream.findFirst(); // Now it executes – and stops after first match
```

---

### Q369. Avoid stateful operations in parallel streams.

```
// WRONG – stateful lambda causes race conditions in parallel
List<Integer> results = new ArrayList<>();
numbers.parallelStream().forEach(n -> results.add(n * 2)); // NOT SAFE

// CORRECT – use collect (thread-safe)
List<Integer> safeResults = numbers.parallelStream()
    .map(n -> n * 2)
    .collect(Collectors.toList());
```

---

### Q370. Profile stream performance.

```
List<Integer> list = IntStream.rangeClosed(1, 10_000_000).boxed().collect(Collectors.toList());

long start = System.nanoTime();
long sum = list.stream().mapToLong(n -> n).sum();
long seqNs = System.nanoTime() - start;

start = System.nanoTime();
long parSum = list.parallelStream().mapToLong(n -> n).sum();
long parNs = System.nanoTime() - start;

System.out.printf("Sequential: %.2fms%n", seqNs / 1e6);
System.out.printf("Parallel:   %.2fms%n", parNs / 1e6);
```

---

### Q371. Optimize groupingBy operations.

```
// Prefer downstream collectors over post-processing
// LESS efficient:
Map<String, List<Employee>> byDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept));
Map<String, Long> counts = new HashMap<>();
byDept.forEach((k, v) -> counts.put(k, (long) v.size()));

// MORE efficient – single pass:
Map<String, Long> efficientCounts = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept, Collectors.counting()));
```

---

### Q372. Understand cost of boxing/unboxing.

```
List<Integer> boxed = IntStream.rangeClosed(1, 1_000_000).boxed().collect(Collectors.toList());

// SLOW – autoboxing in stream
long slowSum = boxed.stream().reduce(0, Integer::sum);

// FAST – use primitive IntStream to avoid boxing
long fastSum = boxed.stream().mapToLong(Integer::longValue).sum();

// FASTEST – use IntStream directly from the start
long fastest = IntStream.rangeClosed(1, 1_000_000).asLongStream().sum();
```

---

## Multi-level Grouping

### Q373. Group by department and then by gender.

```
Map<String, Map<String, List<Employee>>> result = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDept,
        Collectors.groupingBy(Employee::getGender)
    ));
```

---

### Q374. Group by city, then by department.

```
Map<String, Map<String, List<Employee>>> result = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getCity,
        Collectors.groupingBy(Employee::getDept)
    ));
```

---

#### Q375. Multi-level grouping with counting.

```
Map<String, Map<String, Long>> result = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDept,
        Collectors.groupingBy(Employee::getGender, Collectors.counting())
    ));
```

---

#### Q376. Multi-level grouping with averaging.

```
Map<String, Map<String, Double>> result = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDept,
        Collectors.groupingBy(
            Employee::getGender,
            Collectors.averagingDouble(Employee::getSalary)
        )
    ));
```

---

#### Q377. Create nested maps using groupingBy.

```
Map<String, Map<String, Map<String, List<Employee>>>> result = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDept,
        Collectors.groupingBy(
            Employee::getGender,
            Collectors.groupingBy(Employee::getCity)
        )
    ));
```

---

#### Q378. Group employees by department, city, and gender.

```
// Using a composite key string
Map<String, List<Employee>> result = employees.stream()
    .collect(Collectors.groupingBy(
        e -> e.getDept() + "_" + e.getCity() + "_" + e.getGender()
    ));
```

---

### Custom Object Scenarios

#### Q379. Filter products by category and price range.

```
List<Product> result = products.stream()
    .filter(p -> p.getCategory().equals("Electronics"))
    .filter(p -> p.getPrice() >= 500 && p.getPrice() <= 2000)
    .collect(Collectors.toList());
```

---

**Q380. Find top-selling products.**

```
List<Product> topSelling = products.stream()
    .sorted(Comparator.comparingInt(Product::getUnitsSold).reversed())
    .limit(5)
    .collect(Collectors.toList());
```

---

**Q381. Calculate order totals.**

```
Map<String, Double> orderTotals = orders.stream()
    .collect(Collectors.toMap(
        Order::getOrderId,
        o -> o.getItems().stream().mapToDouble(i -> i.getPrice() * i.getQty()).sum()
    ));
```

---

**Q382. Group orders by customer.**

```
Map<String, List<Order>> ordersByCustomer = orders.stream()
    .collect(Collectors.groupingBy(Order::getCustomerId));
```

---

**Q383. Find customers with most orders.**

```
String topCustomer = orders.stream()
    .collect(Collectors.groupingBy(Order::getCustomerId, Collectors.counting()))
    .entrySet().stream()
    .max(Map.Entry.comparingByValue())
    .map(Map.Entry::getKey)
    .orElse("None");
```

---

**Q384. Calculate customer lifetime value.**

```
Map<String, Double> clv = orders.stream()
    .collect(Collectors.groupingBy(
        Order::getCustomerId,
        Collectors.summingDouble(Order::getTotal)
    ));
```

---

**Q385. Find students with highest marks.**

```
Optional<Student> topper = students.stream()
    .max(Comparator.comparingDouble(Student::getMarks));
topper.ifPresent(s -> System.out.println(s.getName() + ": " + s.getMarks()));
```

---

**Q386. Group students by grade and subject.**

```
Map<String, Map<String, List<Student>>> result = students.stream()
    .collect(Collectors.groupingBy(
        Student::getGrade,
        Collectors.groupingBy(Student::getSubject)
    ));
```

---



**Q387. Calculate class averages.**

```
Map<String, Double> classAvg = students.stream()
    .collect(Collectors.groupingBy(
        Student::getGrade,
        Collectors.averagingDouble(Student::getMarks)
    ));
```

---

**Q388. Find students passing all subjects.**

```
// Assuming each Student has a list of subject scores, passing = score >= 40
List<Student> passing = students.stream()
    .filter(s -> s.getSubjectScores().values().stream().allMatch(score -> score >= 40))
    .collect(Collectors.toList());
```

---

## Date & Time Operations

**Q389. Filter dates in a range.**

```
LocalDate start = LocalDate.of(2024, 1, 1);
LocalDate end = LocalDate.of(2024, 6, 30);
List<LocalDate> result = dates.stream()
    .filter(d -> !d.isBefore(start) && !d.isAfter(end))
    .collect(Collectors.toList());
```

---

**Q390. Group events by month.**

```
Map<Month, List<LocalDate>> byMonth = events.stream()
    .collect(Collectors.groupingBy(LocalDate::getMonth));
```

---

**Q391. Find the earliest and latest date.**

```
Optional<LocalDate> earliest = dates.stream().min(LocalDate::compareTo);
Optional<LocalDate> latest = dates.stream().max(LocalDate::compareTo);
```

---

**Q392. Calculate duration between dates.**

```
List<Long> durations = datePairs.stream()
    .map(pair -> ChronoUnit.DAYS.between(pair.getStart(), pair.getEnd()))
    .collect(Collectors.toList());
```

---

**Q393. Filter weekend dates.**

```
List<LocalDate> weekends = dates.stream()
    .filter(d -> d.getDayOfWeek() == DayOfWeek.SATURDAY
        || d.getDayOfWeek() == DayOfWeek.SUNDAY)
    .collect(Collectors.toList());
```

---

**Q394. Group transactions by day/month/year.**

```
// By month
Map<Month, List<Transaction>> byMonth = transactions.stream()
    .collect(Collectors.groupingBy(t -> t.getDate().getMonth()));

// By year
Map<Integer, List<Transaction>> byYear = transactions.stream()
    .collect(Collectors.groupingBy(t -> t.getDate().getYear()));
```

---

## Stream of Streams

### Q395. Flatten stream of streams.

```
Stream<Stream<Integer>> sos = Stream.of(
    Stream.of(1, 2, 3),
    Stream.of(4, 5),
    Stream.of(6, 7, 8)
);
List<Integer> flat = sos.flatMap(s -> s).collect(Collectors.toList());
// [1, 2, 3, 4, 5, 6, 7, 8]
```

---

### Q396. Process nested collections.

```
List<Department> departments = getDepartments();
List<String> allNames = departments.stream()
    .flatMap(dept -> dept.getEmployees().stream())
    .map(Employee::getName)
    .collect(Collectors.toList());
```

---

### Q397. Combine multiple streams.

```
Stream<Integer> s1 = Stream.of(1, 2, 3);
Stream<Integer> s2 = Stream.of(4, 5, 6);
Stream<Integer> s3 = Stream.of(7, 8, 9);

List<Integer> combined = Stream.of(s1, s2, s3)
    .flatMap(s -> s)
    .collect(Collectors.toList());
```

---

### Q398. Merge and deduplicate streams.

```
Stream<String> s1 = Stream.of("apple", "banana", "cherry");
Stream<String> s2 = Stream.of("banana", "date", "cherry");

List<String> merged = Stream.concat(s1, s2)
    .distinct()
    .sorted()
    .collect(Collectors.toList());
// [apple, banana, cherry, date]
```

---

## Error Handling

### Q399. Handle exceptions gracefully in stream pipelines.

```
List<String> inputs = Arrays.asList("10", "abc", "20", "xyz", "30");
List<Integer> results = inputs.stream()
    .flatMap(s -> {
        try {
            return Stream.of(Integer.parseInt(s));
        } catch (NumberFormatException e) {
            return Stream.empty(); // skip invalid
        }
    })
    .collect(Collectors.toList());
// [10, 20, 30]
```

---

**Q400. Use try-catch inside stream operations.**

```
List<String> urls = Arrays.asList("http://a.com", "bad_url", "http://b.com");
List<String> content = urls.stream()
    .map(url -> {
        try {
            return fetchContent(url); // may throw IOException
        } catch (IOException e) {
            return "ERROR: " + url;
        }
    })
    .collect(Collectors.toList());
```

---

## 20. Advanced & Complex Scenarios

### Error Handling

**Q401. Filter out elements that cause exceptions.**

```
List<String> inputs = Arrays.asList("5", "hello", "10", null, "20");
List<Integer> safe = inputs.stream()
    .filter(s -> s != null)
    .flatMap(s -> {
        try {
            return Stream.of(Integer.parseInt(s));
        } catch (NumberFormatException e) {
            return Stream.empty();
        }
    })
    .collect(Collectors.toList());
// [5, 10, 20]
```

---

**Q402. Wrap checked exceptions in unchecked exceptions.**

```
// Helper method to wrap checked exception
static <T, R> Function<T, R> wrap(CheckedFunction<T, R> fn) {
    return t -> {
        try {
            return fn.apply(t);
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    };
}

// Usage in stream:
List<String> lines = filePaths.stream()
    .map(wrap(path -> Files.readString(Paths.get(path))))
    .collect(Collectors.toList());
```

## Stream Best Practices

### Q403. Avoid modifying source during stream operations.

```
List<Integer> numbers = new ArrayList<>(Arrays.asList(1, 2, 3, 4, 5));

// WRONG – modifying source during stream is undefined behavior:
// numbers.stream().forEach(n -> numbers.remove(n)); // ConcurrentModificationException!

// CORRECT – collect to a new list, then reassign:
List<Integer> evens = numbers.stream()
    .filter(n -> n % 2 == 0)
    .collect(Collectors.toList());
```

### Q404. Understand terminal vs intermediate operations.

```
// Intermediate (lazy, return Stream):
// filter(), map(), flatMap(), distinct(), sorted(), peek(), limit(), skip()

// Terminal (eager, trigger execution, return non-Stream):
// forEach(), collect(), reduce(), count(), findFirst(), findAny(),
// anyMatch(), allMatch(), noneMatch(), min(), max(), toArray()

Stream<Integer> stream = Stream.of(1, 2, 3, 4, 5)
    .filter(n -> n > 2)    // intermediate (lazy)
    .map(n -> n * 2);      // intermediate (lazy)

List<Integer> result = stream.collect(Collectors.toList()); // terminal (triggers execution)
```

### Q405. Know when streams can be reused (they can't).

```
Stream<String> stream = Stream.of("a", "b", "c");
stream.forEach(System.out::println); // first use

// stream.forEach(System.out::println); // IllegalStateException: stream already operated upon

// Solution: recreate the stream each time
Supplier<Stream<String>> supplier = () -> Stream.of("a", "b", "c");
supplier.get().forEach(System.out::println);
supplier.get().filter(s -> !s.equals("b")).forEach(System.out::println);
```

#### Q406. Use method references where possible.

```
List<String> names = Arrays.asList("alice", "bob", "charlie");

// Lambda
List<String> upper1 = names.stream().map(s -> s.toUpperCase()).collect(Collectors.toList());

// Method reference (cleaner)
List<String> upper2 = names.stream().map(String::toUpperCase).collect(Collectors.toList());

// Method reference with instance method
names.stream().forEach(System.out::println);
```

---

#### Q407. Prefer collection over reduce when appropriate.

```
// reduce is for single value aggregation
int sum = numbers.stream().reduce(0, Integer::sum);

// collect is for building data structures
List<Integer> list = numbers.stream().filter(n -> n > 2).collect(Collectors.toList());
Map<Boolean, List<Integer>> map = numbers.stream().collect(Collectors.partitioningBy(n -> n % 2 == 0));

// Don't misuse reduce to build collections – it's inefficient due to copying
```

---

#### Q408. Use specialized streams (IntStream) for primitives.

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);

// Using boxed Stream (boxing overhead):
int sum1 = numbers.stream().reduce(0, Integer::sum);

// Using IntStream (no boxing – faster):
int sum2 = numbers.stream().mapToInt(Integer::intValue).sum();

// Even better – use IntStream directly:
int sum3 = IntStream.of(1, 2, 3, 4, 5).sum();
```

---

#### Q409. Avoid forEach for non-side-effect operations.

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);

// WRONG – using forEach to build a collection:
List<Integer> squares = new ArrayList<>();
numbers.stream().forEach(n -> squares.add(n * n)); // side effect

// CORRECT – use map + collect:
List<Integer> correctSquares = numbers.stream()
    .map(n -> n * n)
    .collect(Collectors.toList());
```

---

#### Q410. Understand short-circuiting operations.

```
// Short-circuit terminal ops – stop early when result is known:
// anyMatch, allMatch, noneMatch, findFirst, findAny, limit

boolean found = Stream.of(1, 2, 3, 4, 5)
    .peek(n -> System.out.print(n + " "))
    .anyMatch(n -> n == 3);
// Prints: 1 2 3 – stops at 3

Optional<Integer> first = Stream.of(1, 2, 3, 4, 5)
    .filter(n -> n % 2 == 0)
    .peek(n -> System.out.print("found: " + n))
    .findFirst();
// Prints: found: 2 – stops immediately
```

---

## Tricky Scenarios

### Q411. Handle null elements in streams.

```
List<String> names = Arrays.asList("Alice", null, "Bob", null, "Charlie");

// Filter nulls before processing
List<String> result = names.stream()
    .filter(Objects::nonNull)
    .map(String::toUpperCase)
    .collect(Collectors.toList());
// [ALICE, BOB, CHARLIE]
```

---

### Q412. Work with empty streams.

```
// Empty stream – all terminal ops behave safely
Stream<Integer> empty = Stream.empty();

long count = empty.count(); // 0
Optional<Integer> opt = Stream.<Integer>empty().findFirst(); // Optional.empty
int sum = Stream.<Integer>empty().mapToInt(n -> n).sum(); // 0
```

---

### Q413. Difference between findFirst() and findAny().

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);

// findFirst – always returns the first element in encounter order
Optional<Integer> first = numbers.stream().filter(n -> n > 2).findFirst();
System.out.println(first.get()); // 3 (always)

// findAny – may return any element (non-deterministic in parallel)
Optional<Integer> any = numbers.parallelStream().filter(n -> n > 2).findAny();
System.out.println(any.get()); // 3, 4, or 5 (not guaranteed)
```

---

### Q414. Understanding of lazy evaluation.

```
// Stream operations are lazy – nothing executes until a terminal op
long count = Stream.of("hello", "world", "java", "stream")
    .filter(s -> {
        System.out.println("Filtering: " + s); // will only run as needed
        return s.startsWith("j");
    })
    .count();
// Prints filtering for all elements (count needs to process all)
// But with findFirst() it would stop early
```

---

#### Q415. Side effects in stream operations.

```
// Side effects (changing external state) are discouraged inside stream ops
// Use forEach only for genuine side effects like printing/logging

// BAD – side effect inside map:
List<Integer> sideEffectList = new ArrayList<>();
numbers.stream().map(n -> { sideEffectList.add(n); return n * 2; }).count();

// GOOD – use peek for debug side effects:
numbers.stream()
    .peek(n -> System.out.println("Processing: " + n))
    .map(n -> n * 2)
    .collect(Collectors.toList());
```

---

#### Q416. Stateless vs stateful operations.

```
// Stateless – each element processed independently (filter, map)
List<Integer> stateless = numbers.stream()
    .filter(n -> n > 2) // stateless
    .map(n -> n * 2) // stateless
    .collect(Collectors.toList());

// Stateful – need to see other elements (sorted, distinct, limit)
List<Integer> stateful = numbers.stream()
    .distinct() // stateful – must track seen elements
    .sorted() // stateful – must see all elements
    .limit(5) // stateful – tracks count
    .collect(Collectors.toList());
```

---

#### Q417. Order of operations matters.

```
// filter BEFORE map – fewer elements go through map
List<String> efficient = employees.stream()
    .filter(e -> e.getSalary() > 50000) // reduces elements first
    .map(Employee::getName) // only maps filtered employees
    .collect(Collectors.toList());

// filter AFTER map – all elements mapped, then filtered (less efficient)
List<String> less = employees.stream()
    .map(Employee::getName) // maps all employees
    .filter(n -> n.startsWith("A")) // then filters
    .collect(Collectors.toList());
```

---

#### Q418. When distinct() should be used.

```
// Use distinct() to remove duplicates – uses equals() and hashCode()
List<Integer> numbers = Arrays.asList(1, 2, 2, 3, 3, 4, 1);
List<Integer> unique = numbers.stream().distinct().collect(Collectors.toList());
// [1, 2, 3, 4]

// For objects, ensure equals/hashCode are overridden:
List<Employee> uniqueByName = employees.stream()
    .collect(Collectors.collectingAndThen(
        Collectors.toMap(Employee::getName, e -> e, (e1, e2) -> e1),
        m -> new ArrayList<>(m.values())
    ));
```

#### Q419. Performance implications of sorted().

```
// sorted() is an O(n log n) stateful operation – must buffer all elements
// Avoid if possible, or push it to the end of the pipeline

// Avoid sorting before filter – filter first to reduce elements:
List<Employee> result = employees.stream()
    .filter(e -> e.getSalary() > 50000) // filter first
    .sorted(Comparator.comparing(Employee::getName)) // sort the smaller set
    .collect(Collectors.toList());
```

#### Q420. Understanding collectors composition.

```
// Compose collectors to build complex downstream processing in one pass
Map<String, String> result = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDept,
        Collectors.collectingAndThen(
            Collectors.mapping(Employee::getName, Collectors.joining(", ")),
            names -> "[" + names + "]"
        )
    ));
// {IT=[Alice, Bob], HR=[Charlie]}
```

## Additional Challenge Questions

### Mixed Operations

#### Q421. Find employees with second highest salary per department.

```
Map<String, Optional<Double>> result = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDept,
        Collectors.collectingAndThen(
            Collectors.toList(),
            list -> list.stream()
                .map(Employee::getSalary)
                .distinct()
                .sorted(Collections.reverseOrder())
                .skip(1)
                .findFirst()
        )
    ));
```



**Q422. Find the nth element without using skip.**

```
List<Integer> numbers = Arrays.asList(10, 20, 30, 40, 50);
int n = 3; // 1-based index
int[] index = {0};
Optional<Integer> nth = numbers.stream()
    .filter(num -> ++index[0] == n)
    .findFirst();
System.out.println(nth.orElse(-1)); // 30
```

---

**Q423. Implement running sum using streams.**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
int[] runningSum = {0};
List<Integer> result = numbers.stream()
    .map(n -> runningSum[0] += n)
    .collect(Collectors.toList());
System.out.println(result); // [1, 3, 6, 10, 15]
```

---

**Q424. Find missing numbers in a sequence.**

```
List<Integer> numbers = Arrays.asList(1, 2, 4, 6, 7, 9, 10);
int max = numbers.stream().max(Integer::compareTo).orElse(0);

List<Integer> missing = IntStream.rangeClosed(1, max)
    .filter(n -> !numbers.contains(n))
    .boxed()
    .collect(Collectors.toList());
// [3, 5, 8]
```

---

**Q425. Rotate a list using streams.**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
int k = 2; // rotate right by 2
int size = numbers.size();

List<Integer> rotated = Stream.concat(
    numbers.stream().skip(size - k),
    numbers.stream().limit(size - k)
).collect(Collectors.toList());
// [4, 5, 1, 2, 3]
```

---

**Q426. Implement custom sorting logic.**

```
// Sort employees: IT dept first, then by salary descending
List<Employee> sorted = employees.stream()
    .sorted(Comparator
        .comparing((Employee e) -> e.getDept().equals("IT") ? 0 : 1)
        .thenComparing(Comparator.comparingDouble(Employee::getSalary).reversed()))
    .collect(Collectors.toList());
```

---

**Q427. Find elements at even indices.**

```
List<String> names = Arrays.asList("A", "B", "C", "D", "E");
List<String> evenIndexElements = IntStream.range(0, names.size())
    .filter(i -> i % 2 == 0)
    .mapToObj(names::get)
    .collect(Collectors.toList());
// [A, C, E] (indices 0, 2, 4)
```

---

#### Q428. Find elements at odd indices.

```
List<String> names = Arrays.asList("A", "B", "C", "D", "E");
List<String> oddIndexElements = IntStream.range(0, names.size())
    .filter(i -> i % 2 != 0)
    .mapToObj(names::get)
    .collect(Collectors.toList());
// [B, D] (indices 1, 3)
```

---

#### Q429. Chunk a list into sublists of size N.

```
List<Integer> numbers = IntStream.rangeClosed(1, 10).boxed().collect(Collectors.toList());
int chunkSize = 3;

List<List<Integer>> chunks = IntStream.range(0, (numbers.size() + chunkSize - 1) / chunkSize)
    .mapToObj(i -> numbers.subList(
        i * chunkSize,
        Math.min((i + 1) * chunkSize, numbers.size())
    ))
    .collect(Collectors.toList());
// [[1,2,3], [4,5,6], [7,8,9], [10]]
```

---

#### Q430. Implement zip operation (combine two lists).

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
List<Integer> ages = Arrays.asList(30, 25, 35);

List<String> zipped = IntStream.range(0, Math.min(names.size(), ages.size()))
    .mapToObj(i -> names.get(i) + " -> " + ages.get(i))
    .collect(Collectors.toList());
// [Alice -> 30, Bob -> 25, Charlie -> 35]
```

---

## String Challenges

#### Q431. Remove all special characters from strings.

```
List<String> inputs = Arrays.asList("hello!", "world@#", "java$stream");
List<String> cleaned = inputs.stream()
    .map(s -> s.replaceAll("[^a-zA-Z0-9]", ""))
    .collect(Collectors.toList());
// [hello, world, javastream]
```

---

#### Q432. Find the longest common prefix.

```
List<String> words = Arrays.asList("flower", "flow", "flight");
String prefix = words.stream()
    .reduce((a, b) -> {
        int i = 0;
        while (i < a.length() && i < b.length() && a.charAt(i) == b.charAt(i)) i++;
        return a.substring(0, i);
    })
    .orElse("");
System.out.println(prefix); // "fl"
```

---

#### Q433. Group anagrams together.

```
List<String> words = Arrays.asList("eat", "tea", "tan", "ate", "nat", "bat");
Map<String, List<String>> anagrams = words.stream()
    .collect(Collectors.groupingBy(
        s -> s.chars().sorted()
            .collect(StringBuilder::new, StringBuilder::appendCodePoint, StringBuilder::append)
            .toString()
    ));
// {aet=[eat, tea, ate], ant=[tan, nat], abt=[bat]}
```

---

#### Q434. Sort strings by number of vowels.

```
List<String> words = Arrays.asList("apple", "bee", "stream", "io", "java");
List<String> sorted = words.stream()
    .sorted(Comparator.comparingLong(s ->
        s.chars().filter(c -> "aeiouAEIOU".indexOf(c) != -1).count()))
    .collect(Collectors.toList());
```

---

#### Q435. Find strings with unique characters only.

```
List<String> words = Arrays.asList("abcde", "hello", "stream", "java");
List<String> unique = words.stream()
    .filter(s -> s.chars().distinct().count() == s.length())
    .collect(Collectors.toList());
// [abcde, stream]
```

---

#### Q436. Implement string compression.

```
// Compress "aaabbbcc" -> "a3b3c2"
String input = "aaabbbcc";
String compressed = input.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(c -> c, LinkedHashMap::new, Collectors.counting()))
    .entrySet().stream()
    .map(e -> e.getKey() + "" + e.getValue())
    .collect(Collectors.joining());
System.out.println(compressed); // a3b3c2
```

---

#### Q437. Check if strings are rotations of each other.

```
String s1 = "abcde";
String s2 = "cdeab";
boolean isRotation = s1.length() == s2.length() && (s1 + s1).contains(s2);
System.out.println(isRotation); // true
```

---

**Q438. Find all substrings of a string.**

```
String str = "abc";
List<String> substrings = IntStream.range(0, str.length())
    .boxed()
    .flatMap(i -> IntStream.rangeClosed(i + 1, str.length())
        .mapToObj(j -> str.substring(i, j)))
    .collect(Collectors.toList());
// [a, ab, abc, b, bc, c]
```

---

## Number Challenges

**Q439. Find prime numbers in a range.**

```
List<Integer> primes = IntStream.rangeClosed(2, 50)
    .filter(n -> IntStream.rangeClosed(2, (int) Math.sqrt(n)).allMatch(i -> n % i != 0))
    .boxed()
    .collect(Collectors.toList());
// [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
```

---

**Q440. Generate Fibonacci sequence using streams.**

```
Stream.iterate(new long[]{0, 1}, f -> new long[]{f[1], f[0] + f[1]})
    .limit(10)
    .map(f -> f[0])
    .forEach(System.out::println);
// 0, 1, 1, 2, 3, 5, 8, 13, 21, 34
```

---

**Q441. Find Armstrong numbers.**

```
// Armstrong: 153 = 1^3 + 5^3 + 3^3
List<Integer> armstrongs = IntStream.rangeClosed(1, 999)
    .filter(n -> {
        int digits = String.valueOf(n).length();
        int sum = String.valueOf(n).chars()
            .map(c -> (int) Math.pow(c - '0', digits))
            .sum();
        return sum == n;
    })
    .boxed()
    .collect(Collectors.toList());
// [1, 2, 3, 4, 5, 6, 7, 8, 9, 153, 370, 371, 407]
```

---

**Q442. Check if a number is perfect number.**

```
int n = 28;
boolean isPerfect = IntStream.rangeClosed(1, n / 2)
    .filter(i -> n % i == 0)
    .sum() == n;
System.out.println(isPerfect); // true (28 = 1+2+4+7+14)
```

---

**Q443. Find GCD of numbers in a list.**

```
List<Integer> numbers = Arrays.asList(12, 18, 24, 36);
int gcd = numbers.stream()
    .reduce((a, b) -> {
        while (b != 0) { int t = b; b = a % b; a = t; }
        return a;
    })
    .orElse(0);
System.out.println(gcd); // 6
```

---

**Q444. Find LCM of numbers in a list.**

```
List<Integer> numbers = Arrays.asList(4, 6, 8);
int lcm = numbers.stream()
    .reduce(1, (a, b) -> {
        int g = a;
        int temp = b;
        while (temp != 0) { int t = temp; temp = g % temp; g = t; }
        return a / g * b; // LCM = a*b/gcd
    });
System.out.println(lcm); // 24
```

---

**Q445. Convert decimal to binary using streams.**

```
int decimal = 13;
String binary = IntStream.iterate(decimal, n -> n > 0, n -> n / 2)
    .map(n -> n % 2)
    .mapToObj(Integer::toString)
    .collect(Collectors.joining());
String reversed = new StringBuilder(binary).reverse().toString();
System.out.println(reversed); // 1101
```

---

**Q446. Find factors of a number.**

```
int n = 36;
List<Integer> factors = IntStream.rangeClosed(1, n)
    .filter(i -> n % i == 0)
    .boxed()
    .collect(Collectors.toList());
// [1, 2, 3, 4, 6, 9, 12, 18, 36]
```

---

**Q447. Check if numbers form an arithmetic sequence.**

```
List<Integer> numbers = Arrays.asList(2, 5, 8, 11, 14);
int diff = numbers.get(1) - numbers.get(0);
boolean isArithmetic = IntStream.range(1, numbers.size())
    .allMatch(i -> numbers.get(i) - numbers.get(i - 1) == diff);
System.out.println(isArithmetic); // true
```

---

**Q448. Check if numbers form a geometric sequence.**

```
List<Integer> numbers = Arrays.asList(2, 6, 18, 54);
double ratio = (double) numbers.get(1) / numbers.get(0);
boolean isGeometric = IntStream.range(1, numbers.size())
    .allMatch(i -> (double) numbers.get(i) / numbers.get(i - 1) == ratio);
System.out.println(isGeometric); // true (ratio = 3)
```

---

## Collection Challenges

**Q449. Rotate elements in a list by K positions.**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7);
int k = 3;
int size = numbers.size();
k = k % size;

List<Integer> rotated = Stream.concat(
    numbers.stream().skip(size - k),
    numbers.stream().limit(size - k)
).collect(Collectors.toList());
// [5, 6, 7, 1, 2, 3, 4]
```

---

**Q450. Find the kth largest element.**

```
List<Integer> numbers = Arrays.asList(3, 1, 7, 4, 9, 2, 6);
int k = 2;
Optional<Integer> kthLargest = numbers.stream()
    .distinct()
    .sorted(Collections.reverseOrder())
    .skip(k - 1)
    .findFirst();
System.out.println(kthLargest.get()); // 7
```

---

**Q451. Find the kth smallest element.**

```
List<Integer> numbers = Arrays.asList(3, 1, 7, 4, 9, 2, 6);
int k = 3;
Optional<Integer> kthSmallest = numbers.stream()
    .distinct()
    .sorted()
    .skip(k - 1)
    .findFirst();
System.out.println(kthSmallest.get()); // 4
```

---

**Q452. Implement sliding window operations.**

```
List<Integer> numbers = Arrays.asList(1, 3, 5, 7, 9, 11);
int windowSize = 3;

List<Integer> slidingWindowSums = IntStream.rangeClosed(0, numbers.size() - windowSize)
    .mapToObj(i -> numbers.subList(i, i + windowSize))
    .map(window -> window.stream().mapToInt(n -> n).sum())
    .collect(Collectors.toList());
// [9, 15, 21, 27]
```

---

**Q453. Find majority element (appearing > n/2 times).**

```
List<Integer> numbers = Arrays.asList(3, 3, 4, 2, 3, 3, 3);
int n = numbers.size();
Optional<Integer> majority = numbers.stream()
    .collect(Collectors.groupingBy(i -> i, Collectors.counting()))
    .entrySet().stream()
    .filter(e -> e.getValue() > n / 2)
    .map(Map.Entry::getKey)
    .findFirst();
System.out.println(majority.orElse(-1)); // 3
```

---

**Q454. Find all triplets that sum to zero.**

```
List<Integer> numbers = Arrays.asList(-4, -1, -1, 0, 1, 2);
List<String> triplets = numbers.stream()
    .flatMap(a -> numbers.stream()
        .filter(b -> b > a)
        .flatMap(b -> numbers.stream()
            .filter(c -> c > b && a + b + c == 0)
            .map(c -> "(" + a + ", " + b + ", " + c + ")")
        ))
    .collect(Collectors.toList());
// [(-1,-1,2), (-1,0,1)]
```

---

**Q455. Implement custom partitioning logic.**

```
// Partition into 3 groups: low (<30), mid (30-60), high (>60)
Map<String, List<Integer>> partitioned = numbers.stream()
    .collect(Collectors.groupingBy(n -> {
        if (n < 30) return "Low";
        else if (n <= 60) return "Mid";
        else return "High";
    }));
```

---

**Q456. Find longest consecutive sequence.**

```
List<Integer> numbers = Arrays.asList(100, 4, 200, 1, 3, 2);
Set<Integer> set = new HashSet<>(numbers);
int longest = numbers.stream()
    .filter(n -> !set.contains(n - 1)) // only start of sequences
    .mapToInt(start -> {
        int len = 0;
        while (set.contains(start + len)) len++;
        return len;
    })
    .max()
    .orElse(0);
System.out.println(longest); // 4 (sequence: 1,2,3,4)
```

---

#### Q457. Implement LRU cache operations using streams.

```
// Build a frequency map – most recently used (simulate with LinkedHashMap ordering)
int capacity = 3;
List<Integer> accessOrder = Arrays.asList(1, 2, 3, 1, 4, 2, 5);

// Simulate LRU: last 'capacity' distinct accesses
Set<Integer> lruCache = accessOrder.stream()
    .collect(Collectors.toCollection(LinkedHashSet::new)) // maintain insertion order + dedupe
    .stream()
    .skip(Math.max(0, (long)(new LinkedHashSet<>(accessOrder).size()) - capacity))
    .collect(Collectors.toCollection(LinkedHashSet::new));
System.out.println(lruCache);
```

---

#### Q458. Find equilibrium index of an array.

```
// Index where sum of elements to the left equals sum to the right
int[] arr = {1, 7, 3, 6, 5, 6};
int total = IntStream.of(arr).sum();
int[] leftSum = {0};

OptionalInt equilIndex = IntStream.range(0, arr.length)
    .filter(i -> {
        int right = total - leftSum[0] - arr[i];
        boolean eq = leftSum[0] == right;
        leftSum[0] += arr[i];
        return eq;
    })
    .findFirst();

System.out.println(equilIndex.orElse(-1)); // 3 (leftSum=11, rightSum=11)
```

---

### Advanced Employee Questions

#### Q459. Find employees with duplicate names.

```
Map<String, Long> nameFreq = employees.stream()
    .collect(Collectors.groupingBy(Employee::getName, Collectors.counting()));

List<Employee> duplicates = employees.stream()
    .filter(e -> nameFreq.get(e.getName()) > 1)
    .collect(Collectors.toList());
```

---

#### Q460. Find department with highest average salary.



```
String topDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept,
        Collectors.averagingDouble(Employee::getSalary)))
    .entrySet().stream()
    .max(Map.Entry.comparingByValue())
    .map(Map.Entry::getKey)
    .orElse("None");
```

---

**Q461. Find employees earning more than their department average.**

```
Map<String, Double> avgByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept,
        Collectors.averagingDouble(Employee::getSalary)));

List<Employee> aboveAvg = employees.stream()
    .filter(e -> e.getSalary() > avgByDept.get(e.getDept()))
    .collect(Collectors.toList());
```

---

**Q462. Find employees earning less than their department average.**

```
Map<String, Double> avgByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept,
        Collectors.averagingDouble(Employee::getSalary)));

List<Employee> belowAvg = employees.stream()
    .filter(e -> e.getSalary() < avgByDept.get(e.getDept()))
    .collect(Collectors.toList());
```

---

**Q463. Calculate salary percentile for each employee.**

```
List<Double> sortedSalaries = employees.stream()
    .map(Employee::getSalary)
    .sorted()
    .collect(Collectors.toList());

Map<Employee, Double> percentiles = employees.stream()
    .collect(Collectors.toMap(
        e -> e,
        e -> (double) sortedSalaries.indexOf(e.getSalary()) / sortedSalaries.size() * 100
    ));
```

---

**Q464. Find employees with same salary in different departments.**

```
Map<Double, List<Employee>> bySalary = employees.stream()
    .collect(Collectors.groupingBy(Employee::getSalary));

List<Employee> result = bySalary.values().stream()
    .filter(list -> list.stream().map(Employee::getDept).distinct().count() > 1)
    .flatMap(List::stream)
    .collect(Collectors.toList());
```

---

**Q465. Get a map of salary ranges and employee counts.**

```
Map<String, Long> salaryRanges = employees.stream()
    .collect(Collectors.groupingBy(e -> {
        double s = e.getSalary();
        if (s < 30000) return "< 30K";
        else if (s < 60000) return "30K-60K";
        else if (s < 100000) return "60K-100K";
        else return "100K+";
    }, Collectors.counting()));
```

---

**Q466. Find employees who are in top 25% salary bracket.**

```
DoubleSummaryStatistics stats = employees.stream()
    .mapToDouble(Employee::getSalary).summaryStatistics();
double q3 = stats.getMin() + 0.75 * (stats.getMax() - stats.getMin());

List<Employee> top25 = employees.stream()
    .filter(e -> e.getSalary() >= q3)
    .collect(Collectors.toList());
```

---

**Q467. Create a report of department-wise gender distribution.**

```
Map<String, Map<String, Long>> report = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDept,
        Collectors.groupingBy(Employee::getGender, Collectors.counting())
    ));

report.forEach((dept, genderMap) ->
    System.out.println(dept + ": " + genderMap));
```

---

**Q468. Find correlation between age and salary.**

```
// Simple correlation check: group by age bracket and average salary
Map<String, Double> avgSalaryByAgeBracket = employees.stream()
    .collect(Collectors.groupingBy(
        e -> e.getAge() < 30 ? "Young" : e.getAge() < 45 ? "Mid" : "Senior",
        Collectors.averagingDouble(Employee::getSalary)
    ));

avgSalaryByAgeBracket.forEach((bracket, avg) ->
    System.out.println(bracket + " avg salary: " + avg));
```

---

**Q469. Identify salary outliers.**

```
DoubleSummaryStatistics stats = employees.stream()
    .mapToDouble(Employee::getSalary).summaryStatistics();
double mean = stats.getAverage();
double range = stats.getMax() - stats.getMin();
double threshold = mean + 1.5 * (range / 4); // simple outlier bound

List<Employee> outliers = employees.stream()
    .filter(e -> e.getSalary() > threshold)
    .collect(Collectors.toList());
```

---

**Q470. Find employees with longest tenure.**

```
LocalDate today = LocalDate.now();
Optional<Employee> longestTenure = employees.stream()
    .min(Comparator.comparing(e -> ChronoUnit.DAYS.between(e.getJoiningDate(), today)));
// min because earliest joining date = longest tenure

longestTenure.ifPresent(e ->
    System.out.println(e.getName() + " joined " + e.getJoiningDate()));
```

---

## Stream Internals & Theory

### Q471. Explain how peek() works vs forEach().

```
// peek() is intermediate – passes element through unchanged, for side effects (e.g., debug)
List<Integer> result = Stream.of(1, 2, 3, 4, 5)
    .peek(n -> System.out.println("Before: " + n))
    .filter(n -> n % 2 == 0)
    .peek(n -> System.out.println("After: " + n))
    .collect(Collectors.toList());

// forEach() is terminal – consumes the stream
Stream.of(1, 2, 3).forEach(System.out::println);
// peek leaves stream open; forEach closes it
```

---

### Q472. Why can't streams be reused?

```
Stream<String> stream = Stream.of("a", "b", "c");
stream.forEach(System.out::println); // OK

try {
    stream.forEach(System.out::println); // IllegalStateException
} catch (IllegalStateException e) {
    System.out.println("Stream already consumed: " + e.getMessage());
}

// Solution: use a Supplier
Supplier<Stream<String>> supplier = () -> Stream.of("a", "b", "c");
supplier.get().forEach(System.out::println);
supplier.get().forEach(System.out::println); // OK – creates fresh streams
```

---

### Q473. Difference between Collection.stream() and Arrays.stream().

```
// Collection.stream() – works on any Collection (List, Set, etc.)
List<String> list = Arrays.asList("a", "b", "c");
Stream<String> s1 = list.stream();

// Arrays.stream() – works on arrays, supports primitive types directly
String[] arr = {"a", "b", "c"};
Stream<String> s2 = Arrays.stream(arr);

int[] intArr = {1, 2, 3};
IntStream s3 = Arrays.stream(intArr); // returns IntStream, not Stream<Integer>
```

---

### Q474. How does parallel stream split the work?

```
// Parallel streams use ForkJoinPool.commonPool() by default
// They split data using Spliterator into chunks and process concurrently

List<Integer> numbers = IntStream.rangeClosed(1, 8).boxed().collect(Collectors.toList());
numbers.parallelStream()
    .map(n -> {
        System.out.println(Thread.currentThread().getName() + ": " + n);
        return n * 2;
    })
    .collect(Collectors.toList());
// Output shows different threads processing different elements
```

#### Q475. What is spliterator and how does it work?

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);
Spliterator<Integer> spliterator = numbers.spliterator();

System.out.println("Est. size: " + spliterator.estimateSize()); // 6

// Split into two – used by parallel streams
Spliterator<Integer> half = spliterator.trySplit();
half.forEachRemaining(n -> System.out.print("Half1: " + n + " "));
spliterator.forEachRemaining(n -> System.out.print("Half2: " + n + " "));
```

#### Q476. Explain the stream pipeline execution.

```
// A stream pipeline has 3 parts:
// 1. Source – data source (list, array, generator)
// 2. Intermediate operations – lazy, return Stream
// 3. Terminal operation – eager, triggers execution

// Execution is driven by the terminal operation
// Elements are pulled through the pipeline one by one (vertical processing)

long result = Stream.of(1, 2, 3, 4, 5) // source
    .filter(n -> n % 2 == 0)           // intermediate (lazy)
    .map(n -> n * 2)                   // intermediate (lazy)
    .count();                         // terminal – triggers all operations
// Elements processed: 2 (filtered) -> 4 (mapped), then 4 (filtered) -> 16 (mapped)
```

#### Q477. What are characteristics of a collector?

```
// A Collector has 5 components:
// 1. supplier() – creates the mutable result container
// 2. accumulator() – adds a single element to container
// 3. combiner() – merges two containers (used in parallel)
// 4. finisher() – transforms container to final result
// 5. characteristics() – CONCURRENT, UNORDERED, IDENTITY_FINISH

// Example – recreating Collectors.toList():
Collector<String, List<String>, List<String>> toListCollector = Collector.of(
    ArrayList::new, // supplier
    List::add,      // accumulator
    (l1, l2) -> { l1.addAll(l2); return l1; }, // combiner
    Collector.Characteristics.IDENTITY_FINISH // finisher is identity
);
```

#### Q478. How does groupingBy work internally?

```
// groupingBy is essentially:
// 1. Create a HashMap
// 2. For each element, apply classifier to get key
// 3. Add element to the list for that key
// 4. Apply downstream collector to each group's list

// Simplified equivalent:
Map<String, List<Employee>> manual = new HashMap<>();
for (Employee e : employees) {
    manual.computeIfAbsent(e.getDept(), k -> new ArrayList<>()).add(e);
}

// Stream version:
Map<String, List<Employee>> stream = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept));
```

#### Q479. Explain combiner function in reduce.

```
// The 3-arg reduce() is for parallel streams
// combiner merges partial results from different threads

int sum = Stream.of(1, 2, 3, 4, 5)
    .parallel()
    .reduce(
        0, // identity
        (a, b) -> a + b, // accumulator
        (partialSum1, partialSum2) -> partialSum1 + partialSum2 // combiner
    );
System.out.println(sum); // 15
```

#### Q480. What is the difference between reduce and collect?

```
// reduce – produces a single immutable value; each step creates new value
int sum = Stream.of(1, 2, 3, 4, 5).reduce(0, Integer::sum);

// collect – uses a mutable container (no new object per step – efficient for containers)
List<Integer> list = Stream.of(1, 2, 3, 4, 5)
    .collect(Collectors.toList()); // accumulates into ArrayList

// Use reduce for: sum, product, max, min, concatenation
// Use collect for: building List, Set, Map, joining strings
```

### Real Interview Scenarios

#### Q481. Given a sentence, find all words with length > 5 and convert to uppercase.

```
String sentence = "Java Stream API is very powerful and flexible";
List<String> result = Arrays.stream(sentence.split(" "))
    .filter(w -> w.length() > 5)
    .map(String::toUpperCase)
    .collect(Collectors.toList());
// [STREAM, POWERFUL, FLEXIBLE]
```

#### Q482. From a list of transactions, find total amount per category.

```
Map<String, Double> totalByCategory = transactions.stream()
    .collect(Collectors.groupingBy(
        Transaction::getCategory,
        Collectors.summingDouble(Transaction::getAmount)
    ));
```

---

**Q483. Find the 3 most expensive products.**

```
List<Product> top3 = products.stream()
    .sorted(Comparator.comparingDouble(Product::getPrice).reversed())
    .limit(3)
    .collect(Collectors.toList());
```

---

**Q484. Calculate moving average of stock prices.**

```
List<Double> prices = Arrays.asList(100.0, 102.0, 104.0, 103.0, 105.0, 107.0);
int window = 3;

List<Double> movingAvg = IntStream.rangeClosed(window - 1, prices.size() - 1)
    .mapToObj(i -> prices.subList(i - window + 1, i + 1))
    .map(sub -> sub.stream().mapToDouble(Double::doubleValue).average().orElse(0))
    .collect(Collectors.toList());
// [102.0, 103.0, 104.0, 105.0]
```

---

**Q485. Find employees who report to the same manager.**

```
Map<Integer, List<Employee>> byManager = employees.stream()
    .filter(e -> e.getManagerId() != null)
    .collect(Collectors.groupingBy(Employee::getManagerId));
```

---

**Q486. Implement a simple recommendation system using streams.**

```
// Recommend products that the user hasn't bought but similar users have
Set<String> userPurchases = Set.of("apple", "banana");
Map<String, Set<String>> allUserPurchases = getSimilarUserPurchases();

List<String> recommendations = allUserPurchases.values().stream()
    .flatMap(Set::stream)
    .filter(item -> !userPurchases.contains(item))
    .distinct()
    .limit(5)
    .collect(Collectors.toList());
```

---

**Q487. Process and aggregate sensor data.**

```
// Average temperature per sensor per hour
Map<String, Map<Integer, Double>> avgTempBySensorAndHour = sensorReadings.stream()
    .collect(Collectors.groupingBy(
        SensorData::getSensorId,
        Collectors.groupingBy(
            r -> r.getTimestamp().getHour(),
            Collectors.averagingDouble(SensorData::getTemperature)
        )
    ));
```

---

#### Q488. Find peak hours from timestamp data.

```
Map<Integer, Long> requestsPerHour = requests.stream()
    .collect(Collectors.groupingBy(
        r -> r.getTimestamp().getHour(),
        Collectors.counting()
    ));

int peakHour = requestsPerHour.entrySet().stream()
    .max(Map.Entry.comparingByValue())
    .map(Map.Entry::getKey)
    .orElse(-1);
System.out.println("Peak hour: " + peakHour + ":00");
```

---

#### Q489. Analyze log files for error patterns.

```
try (Stream<String> lines = Files.lines(Paths.get("app.log"))) {
    Map<String, Long> errorFrequency = lines
        .filter(line -> line.contains("ERROR"))
        .map(line -> line.replaceAll(".*ERROR\\s+", "").split(":")[0]) // extract error type
        .collect(Collectors.groupingBy(e -> e, Collectors.counting()));

    errorFrequency.entrySet().stream()
        .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
        .forEach(e -> System.out.println(e.getKey() + ": " + e.getValue()));
}
```

---

#### Q490. Calculate customer churn rate.

```
long totalCustomers = customers.stream().count();
long churned = customers.stream()
    .filter(c -> c.getLastOrderDate().isBefore(LocalDate.now().minusMonths(6)))
    .count();

double churnRate = (double) churned / totalCustomers * 100;
System.out.printf("Churn rate: %.2f%%\n", churnRate);
```

---

### Edge Cases & Corner Cases

#### Q491. Handle empty collections.

```
List<Integer> empty = Collections.emptyList();

// All safe – return default values or Optional.empty()
long count = empty.stream().count(); // 0
Optional<Integer> max = empty.stream().max(Integer::compareTo); // Optional.empty
int sum = empty.stream().mapToInt(n -> n).sum(); // 0
List<Integer> list = empty.stream().collect(Collectors.toList()); // []
```

---

#### Q492. Handle single-element collections.

```
List<Integer> single = Collections.singletonList(42);

int max = single.stream().mapToInt(n -> n).max().getAsInt(); // 42
int min = single.stream().mapToInt(n -> n).min().getAsInt(); // 42
double avg = single.stream().mapToInt(n -> n).average().getAsDouble(); // 42.0
```

---

#### Q493. Handle very large collections.

```
// Use parallel streams for CPU-bound operations on large data
long primeCount = LongStream.rangeClosed(2, 10_000_000L)
    .parallel()
    .filter(n -> LongStream.rangeClosed(2, (long) Math.sqrt(n)).allMatch(i -> n % i != 0))
    .count();
System.out.println("Primes up to 10M: " + primeCount);
```

---

#### Q494. Deal with null elements in streams.

```
List<String> names = Arrays.asList("Alice", null, "Bob", null);

// Option 1: filter out nulls
List<String> noNulls = names.stream()
    .filter(Objects::nonNull)
    .collect(Collectors.toList());

// Option 2: replace nulls with default
List<String> withDefault = names.stream()
    .map(n -> n != null ? n : "Unknown")
    .collect(Collectors.toList());
```

---

#### Q495. Handle concurrent modifications.

```
// Never modify the source collection during streaming
List<Integer> numbers = new ArrayList<>(Arrays.asList(1, 2, 3, 4, 5));

// WRONG:
// numbers.stream().forEach(n -> { if (n % 2 == 0) numbers.remove(n); }); // throws CME

// CORRECT – removeIf (thread-safe):
numbers.removeIf(n -> n % 2 == 0);
// Or collect a new list:
List<Integer> odds = numbers.stream().filter(n -> n % 2 != 0).collect(Collectors.toList());
```

---

#### Q496. Process infinite streams safely.



```
// Always use limit() or short-circuiting terminal ops on infinite streams
Stream<Integer> infinite = Stream.iterate(1, n -> n + 1);

// SAFE – limit terminates the stream
List<Integer> first10 = infinite.limit(10).collect(Collectors.toList());

// SAFE – findFirst terminates early
Optional<Integer> firstOver100 = Stream.iterate(1, n -> n + 1)
    .filter(n -> n > 100)
    .findFirst();
System.out.println(firstOver100.get()); // 101

// DANGEROUS – never call count(), forEach() etc. on infinite streams without limit
```

---

#### Q497. Handle streams with all identical elements.

```
List<Integer> allSame = Arrays.asList(5, 5, 5, 5, 5);

long distinctCount = allSame.stream().distinct().count(); // 1
int sum = allSame.stream().mapToInt(n -> n).sum(); // 25
int max = allSame.stream().mapToInt(n -> n).max().getAsInt(); // 5
int min = allSame.stream().mapToInt(n -> n).min().getAsInt(); // 5
```

---

#### Q498. Work with streams containing only nulls.

```
List<String> allNulls = Arrays.asList(null, null, null);

// Safe – filter nulls first
long count = allNulls.stream().filter(Objects::nonNull).count(); // 0

// Risky – map/reduce on nulls will throw NPE
// allNulls.stream().map(String::toUpperCase)... // NullPointerException!

List<String> safe = allNulls.stream()
    .filter(Objects::nonNull)
    .collect(Collectors.toList()); // []
```

---

#### Q499. Process interleaved data streams.

```
// Merge two streams maintaining order interleaving
Stream<Integer> odds = Stream.of(1, 3, 5, 7, 9);
Stream<Integer> evens = Stream.of(2, 4, 6, 8, 10);

// Simple concat + sort:
List<Integer> merged = Stream.concat(odds, evens)
    .sorted()
    .collect(Collectors.toList());
// [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

---

#### Q500. Handle streams with irregular data patterns.

```
// Mixed type data – safely convert and handle irregularities
List<Object> mixedData = Arrays.asList(1, "two", 3.0, "four", 5, null);

// Extract only integers safely
List<Integer> integers = mixedData.stream()
    .filter(o -> o instanceof Integer)
    .map(o -> (Integer) o)
    .collect(Collectors.toList());
// [1, 5]

// Extract only strings safely
List<String> strings = mixedData.stream()
    .filter(o -> o instanceof String)
    .map(o -> (String) o)
    .collect(Collectors.toList());
// [two, four]
```

## Quick Reference — Stream API Cheat Sheet

| Operation                  | Type         | Description               | Example                                         |
|----------------------------|--------------|---------------------------|-------------------------------------------------|
| <code>filter(pred)</code>  | Intermediate | Keep matching elements    | <code>.filter(n -&gt; n &gt; 0)</code>          |
| <code>map(fn)</code>       | Intermediate | Transform each element    | <code>.map(String::toUpperCase)</code>          |
| <code>flatMap(fn)</code>   | Intermediate | Flatten nested streams    | <code>.flatMap(List::stream)</code>             |
| <code>distinct()</code>    | Intermediate | Remove duplicates         | <code>.distinct()</code>                        |
| <code>sorted()</code>      | Intermediate | Sort elements             | <code>.sorted(Comparator.naturalOrder())</code> |
| <code>limit(n)</code>      | Intermediate | Take first N elements     | <code>.limit(10)</code>                         |
| <code>skip(n)</code>       | Intermediate | Skip first N elements     | <code>.skip(5)</code>                           |
| <code>peek(fn)</code>      | Intermediate | Debug side-effect         | <code>.peek(System.out::println)</code>         |
| <code>collect(c)</code>    | Terminal     | Gather into collection    | <code>.collect(Collectors.toList())</code>      |
| <code>forEach(fn)</code>   | Terminal     | Consume each element      | <code>.forEach(System.out::println)</code>      |
| <code>count()</code>       | Terminal     | Count elements            | <code>.count()</code>                           |
| <code>reduce(id,fn)</code> | Terminal     | Aggregate to single value | <code>.reduce(0, Integer::sum)</code>           |
| <code>findFirst()</code>   | Terminal     | First element (Optional)  | <code>.findFirst()</code>                       |
| <code>anyMatch(p)</code>   | Terminal     | Any match? (boolean)      | <code>.anyMatch(n -&gt; n &gt; 0)</code>        |
| <code>allMatch(p)</code>   | Terminal     | All match? (boolean)      | <code>.allMatch(n -&gt; n &gt; 0)</code>        |
| <code>noneMatch(p)</code>  | Terminal     | None match? (boolean)     | <code>.noneMatch(Objects::isNull)</code>        |
| <code>min(cmp)</code>      | Terminal     | Minimum element           | <code>.min(Comparator.naturalOrder())</code>    |
| <code>max(cmp)</code>      | Terminal     | Maximum element           | <code>.max(Comparator.naturalOrder())</code>    |
| <code>toArray()</code>     | Terminal     | Convert to array          | <code>.toArray(String[]::new)</code>            |

Good luck with your Java Stream API interviews!